COMP0222 Simultaneous Localisation and Mapping

Lecture 15: 2D Occupancy Grid Mapping

Simon Julier

s.julier@ucl.ac.uk

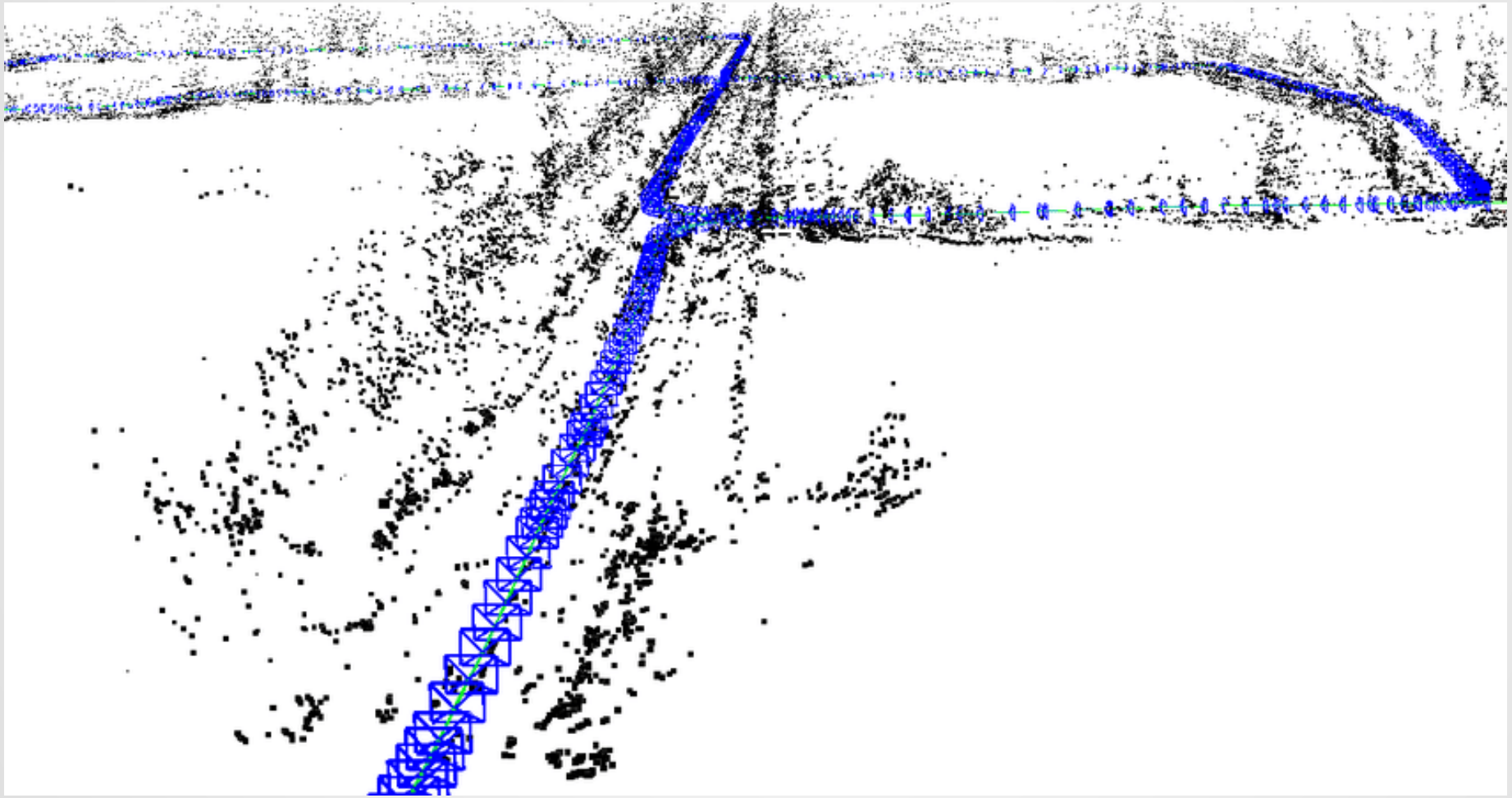
Sajad Saeedi

s.saeedi@ucl.ac.uk

Motivation

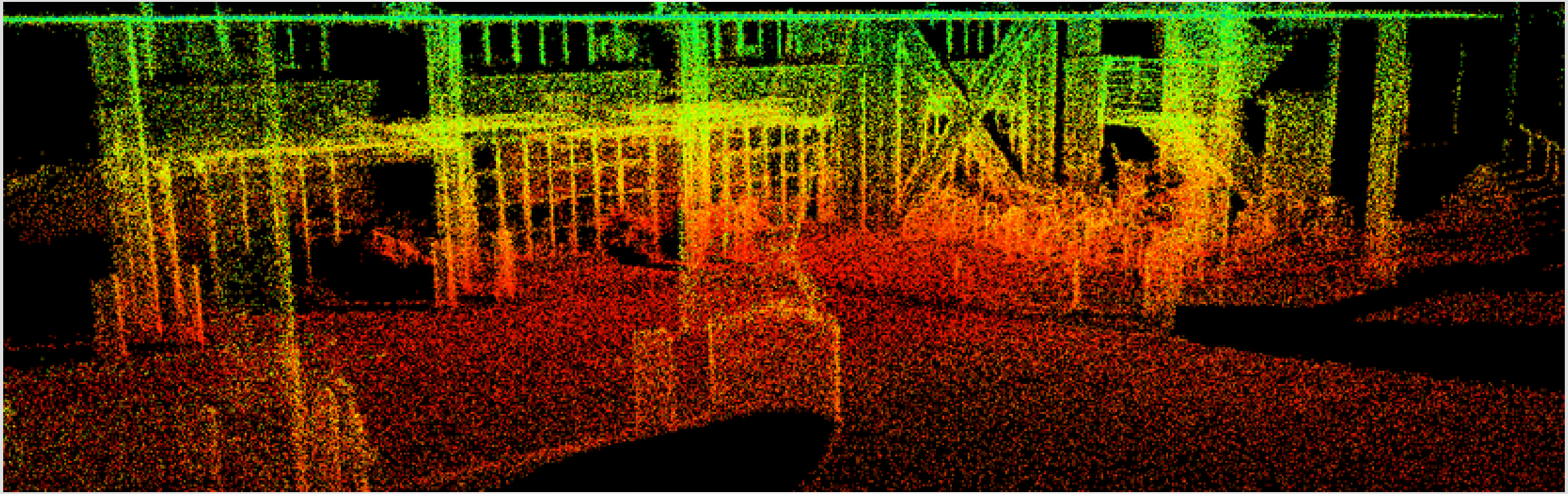
- **All the SLAM algorithms we've seen so far are based on *points***

Sparse Point Cloud



From "Feature-based Visual Odometry Prior for Real-Time Semi-Dense Stereo SLAM", Krombach et al 2018

Dense Point Cloud



From "LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping", Shan et al. 2020

Limitations of Sparse and Dense Point Clouds

- Although these can provide accurate localization and geometry, they might be inefficient tools for planning and navigation
  - If you want to work out where the robot can go, you have to do complicated 3D geometric analysis
- They are also not helpful for exploration
  - If you have a part of the map which is empty, is it because nothing is there, or because it's not been visited yet?

Occupancy Volume Representations

- Rather than map points, we can more efficiently map the space
- It does not store any information about individual points at all
- This is often called an occupancy grid representation and can be 2D or 3D

2D Occupancy Grid Representation

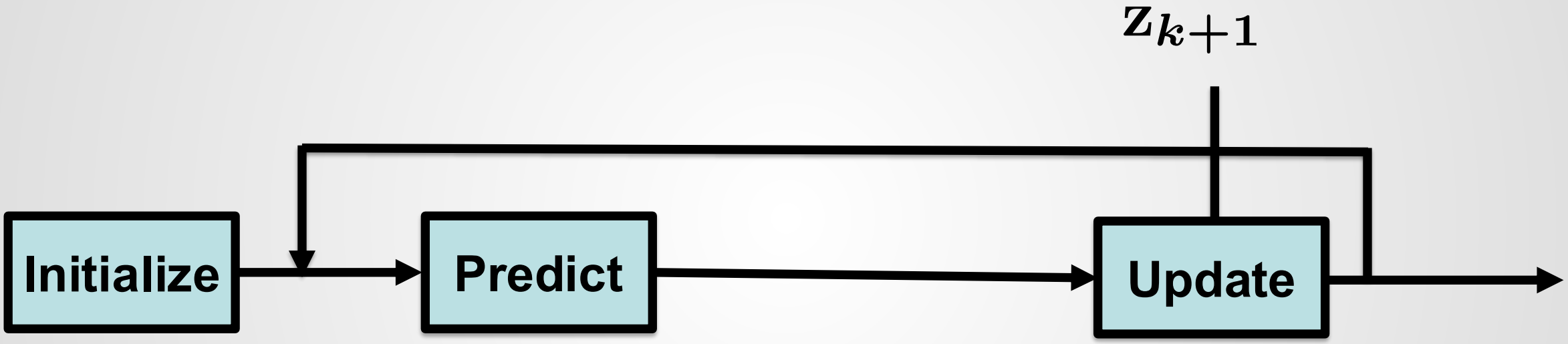
Grid SLAM

- To move to a SLAM solution, we need to estimate both the platform poses and the map at the same time

$$\Pr\left(\mathbf{x}_k, O \middle| \mathbf{x}_0, \mathbf{u}_{1:k}, \mathbf{z}_{1:k}\right)$$

- To do this, we'll use our familiar recursive structure which includes the platform prediction

Probabilistic Occupancy Grid Mapping



$\Pr(O|\mathbf{x}_0)$ $\Pr(\mathbf{x}_{1:k+1}, O|\mathbf{x}_0, \mathbf{u}_{1:k+1}, \mathbf{z}_{1:k})$ $\Pr(\mathbf{x}_{1:k+1}, O|\mathbf{x}_0, \mathbf{u}_{1:k+1}, \mathbf{z}_{1:k+1})$

Structure

- Deterministic Occupancy Grids
- Updating Deterministic Occupancy Grids in the Ideal Case
- Probabilistic Occupancy Grids
- Updating Probabilistic Occupancy Grids with Noisy Sensors
- Scan Matching, Odometry and Occupancy Grids
- Particle Filters and FastSLAM
- Suboptimal Particle and Factor Graph Approaches

Deterministic Occupancy Grids

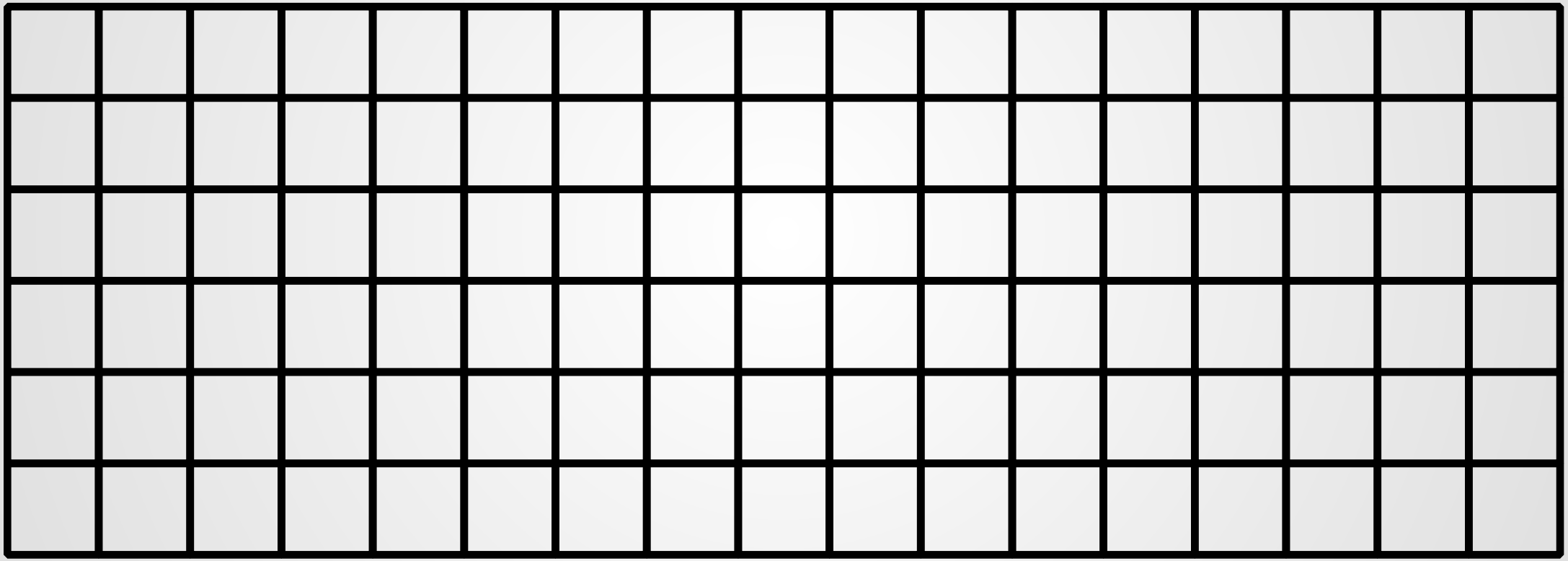
- **Deterministic Occupancy Grids**
- ***Updating Deterministic Occupancy Grids in the Ideal Case***
- ***Probabilistic Occupancy Grids***
- ***Updating Probabilistic Occupancy Grids with Noisy Sensors***
- ***Scan Matching, Odometry and Occupancy Grids***
- ***Particle Filters and FastSLAM***
- ***Suboptimal Particle and Factor Graph Approaches***

Occupancy Grids

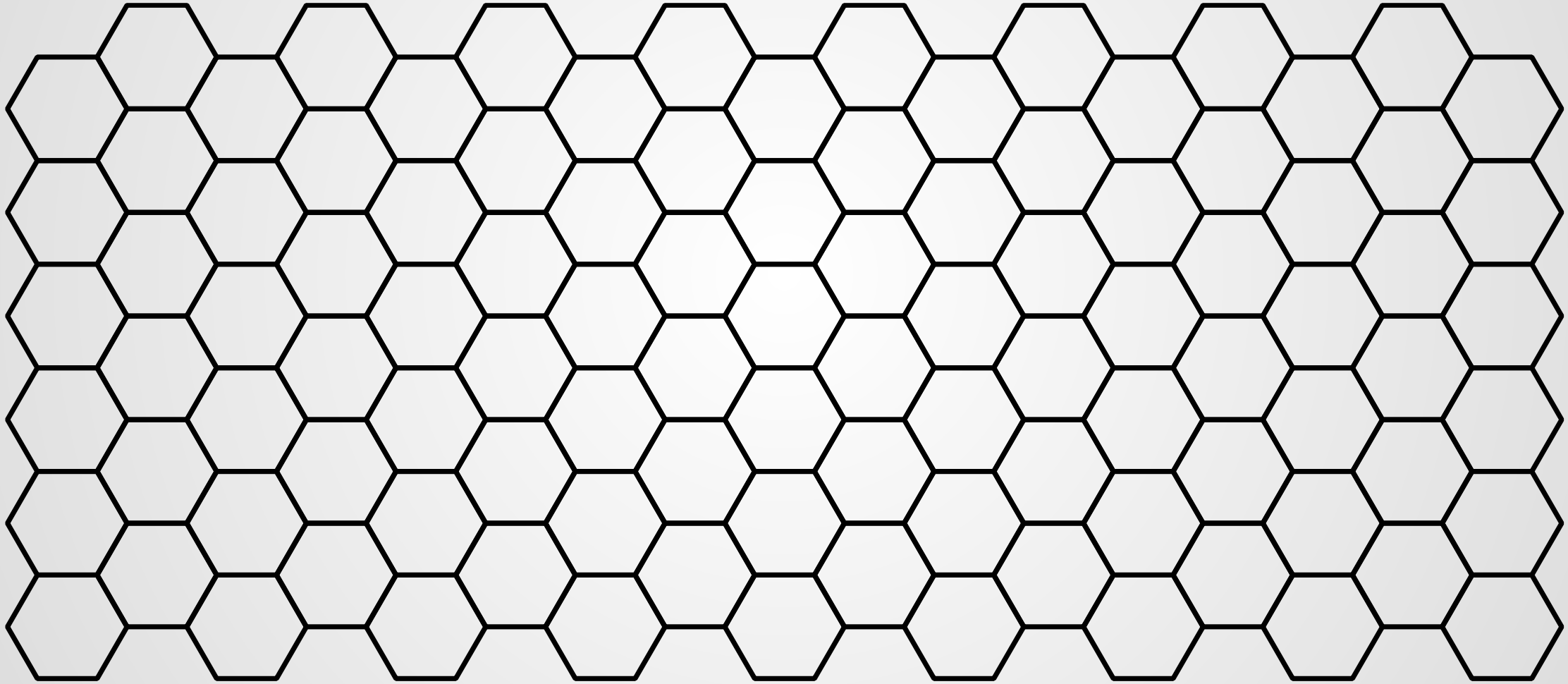
- The environment is decomposed into a mutually exclusive and non-exhaustive set of cells

- This means that:
  - None of the cells overlap one another
  - The union of the cells covers the entire space

Square Grid Map



Rectangular Grid Map



Occupancy Grids

- An occupancy grid is written as O
- It consists of the set of N cells

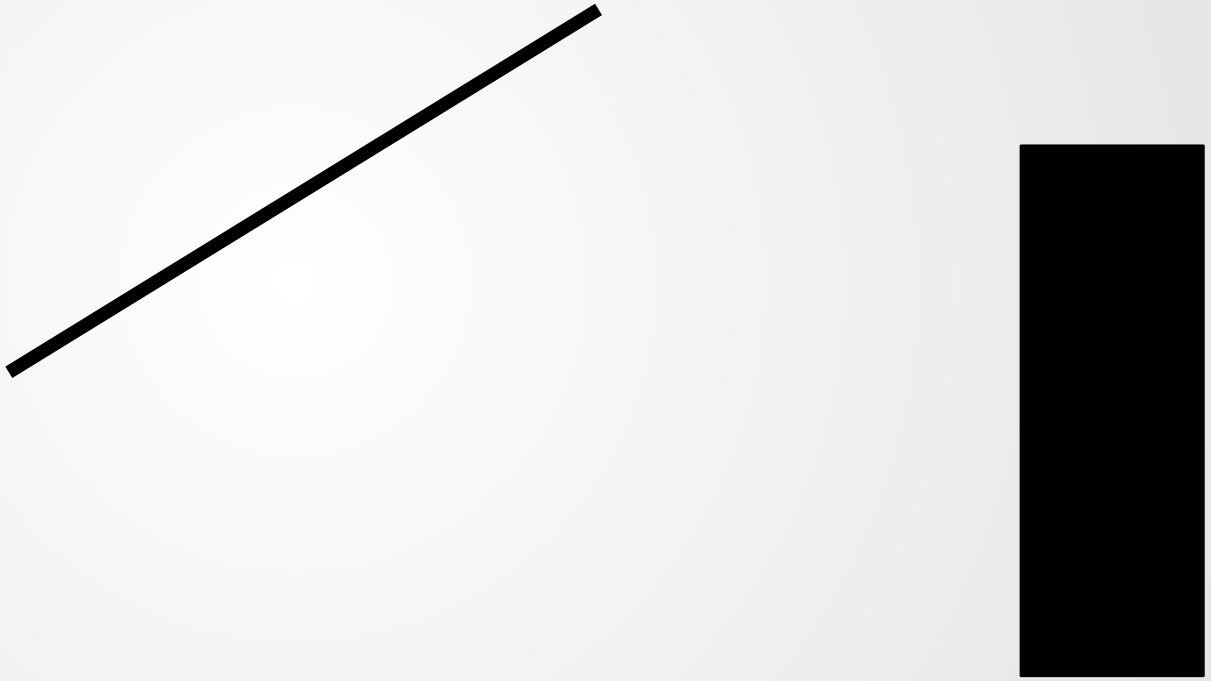
$$O = \{C_1, C_2, \ldots, C_N\}$$

Occupancy Grids

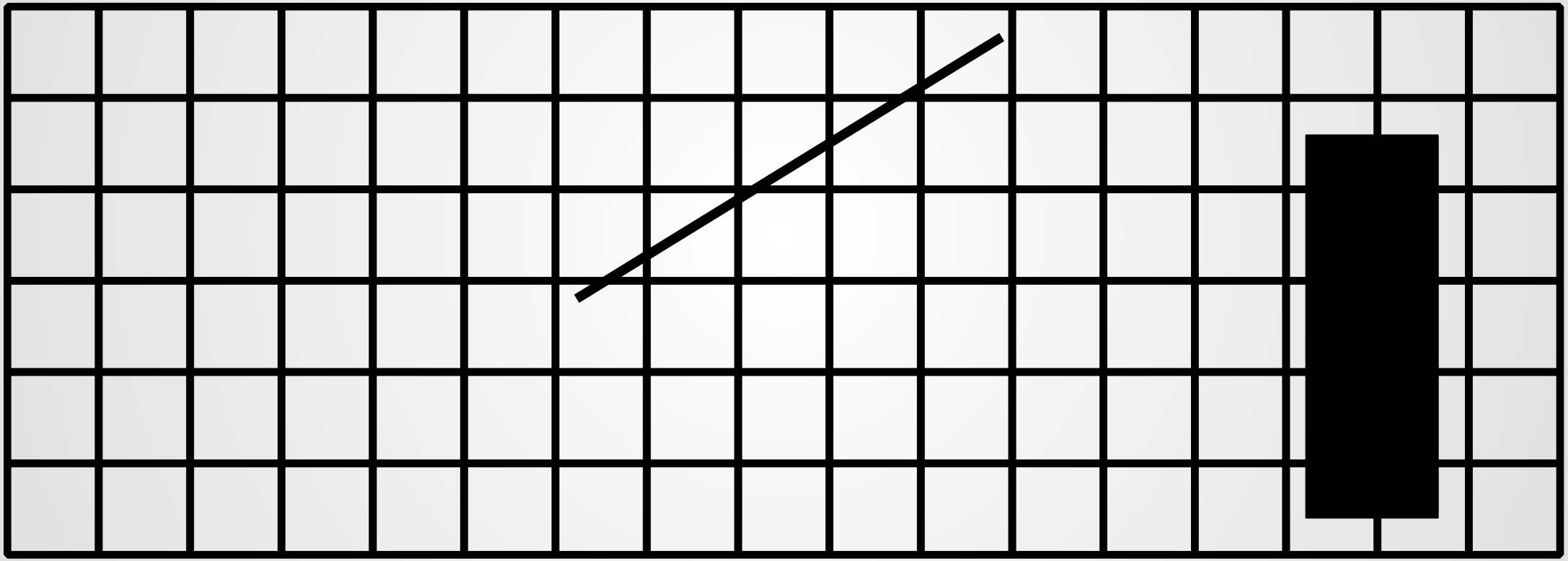
- **Each cell has a state**

$$S\left(C_n\right) = \begin{cases} 0 & C_n \text{ is occupied} \\ 1 & C_n \text{ is free} \end{cases}$$

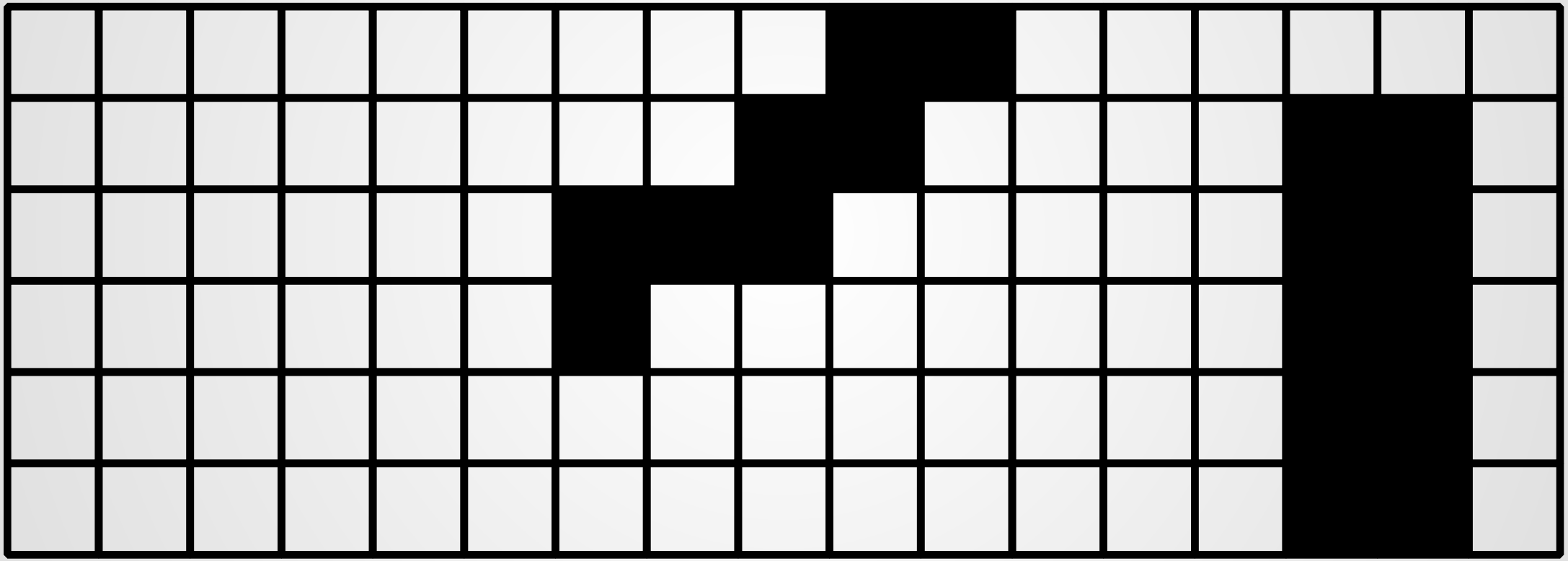
Square Grid Map



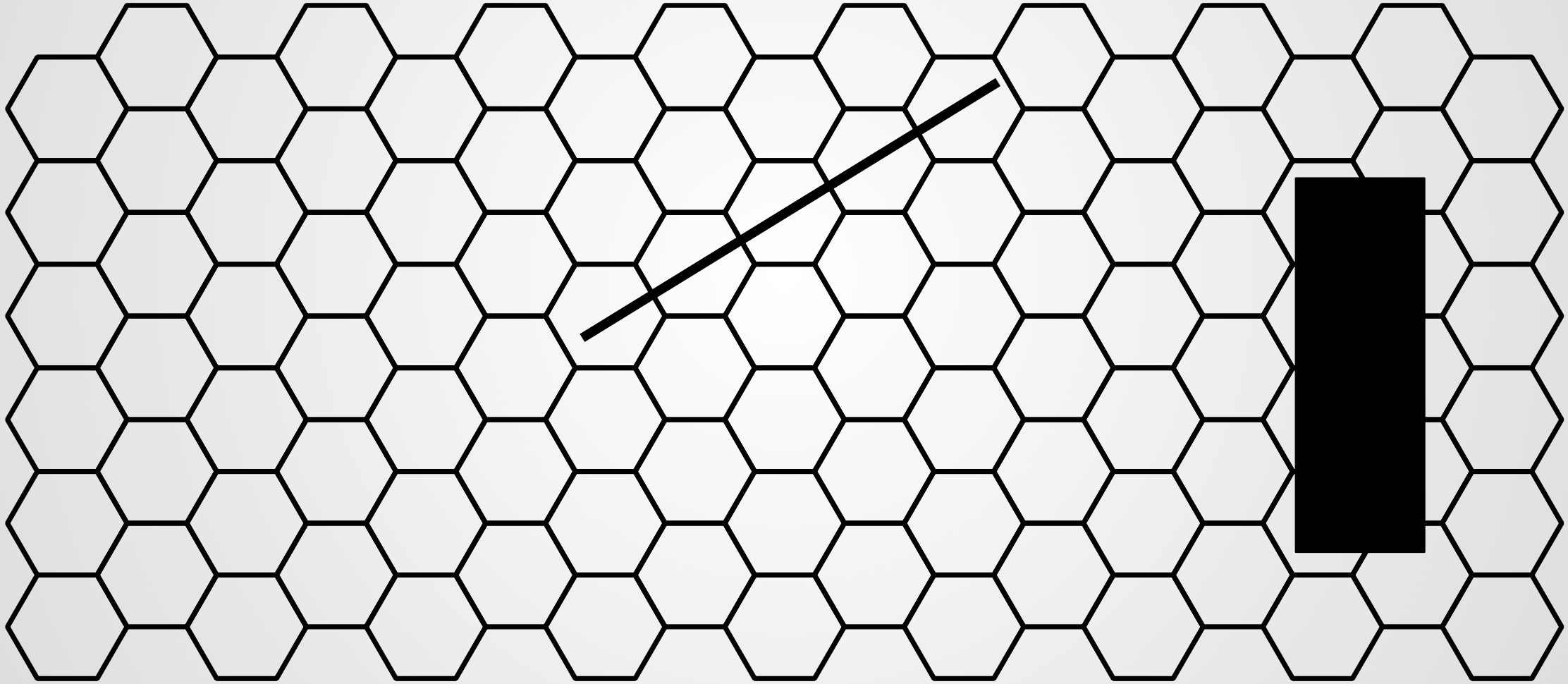
Square Grid Map



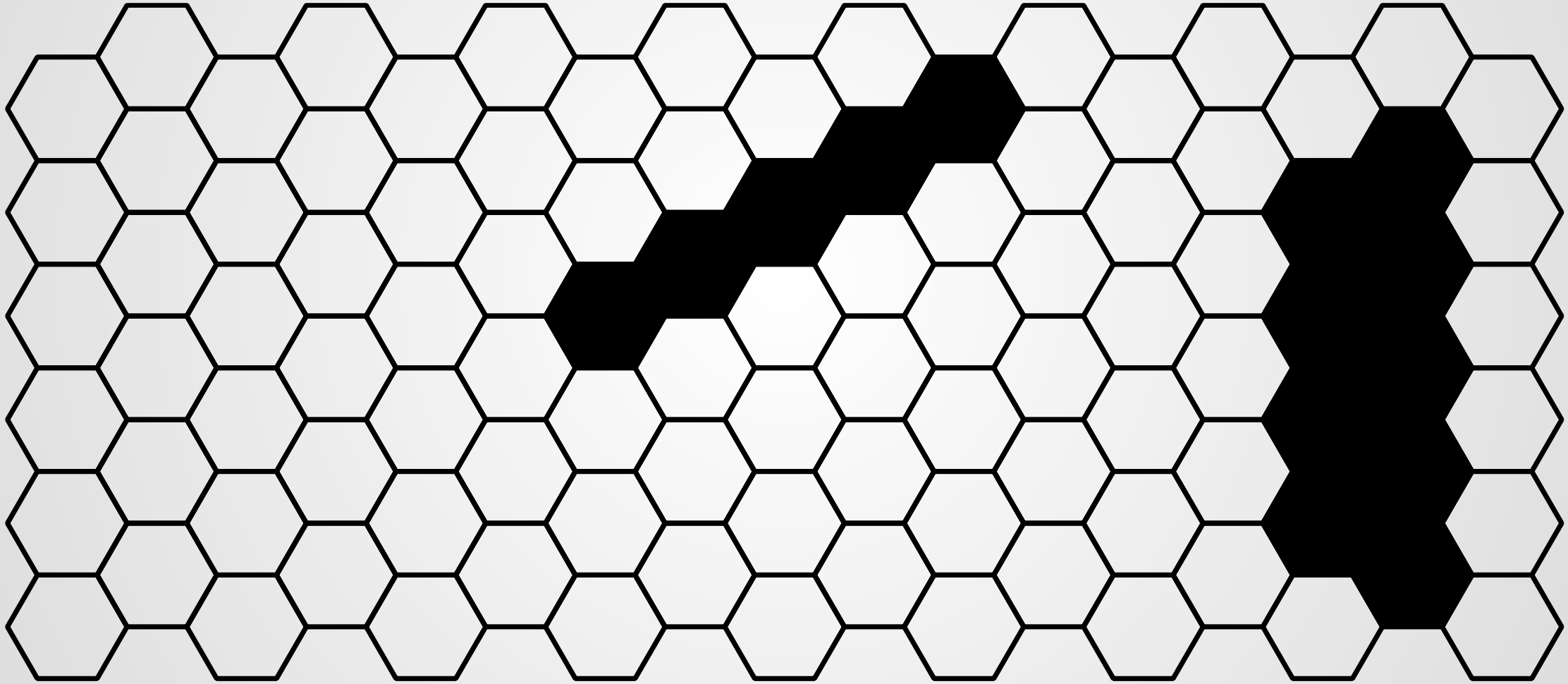
Square Grid Map



Rectangular Grid Map



Rectangular Grid Map



Updating Deterministic Occupancy Grids in the Ideal Case

- *Deterministic Occupancy Grids*
- Updating Deterministic Occupancy Grids in the Ideal Case
- *Probabilistic Occupancy Grids*
- *Updating Probabilistic Occupancy Grids with Noisy Sensors*
- *Scan Matching, Odometry and Occupancy Grids*
- *Particle Filters and FastSLAM*
- *Suboptimal Particle and Factor Graph Approaches*

Occupancy Grid Example

- **Let's first consider the perfect case:**
  - **We know *exactly* where the platform is**
  - **The sensor records distance along a ray with *no measurement error***
- **The occupancy grid:**
  - **Is rectangular**
  - **Each cell is identified by a coordinate $n = (i, j)$**
  - **Each cell has the same width c_w and the same height c_h**

Updating an Occupancy Grid

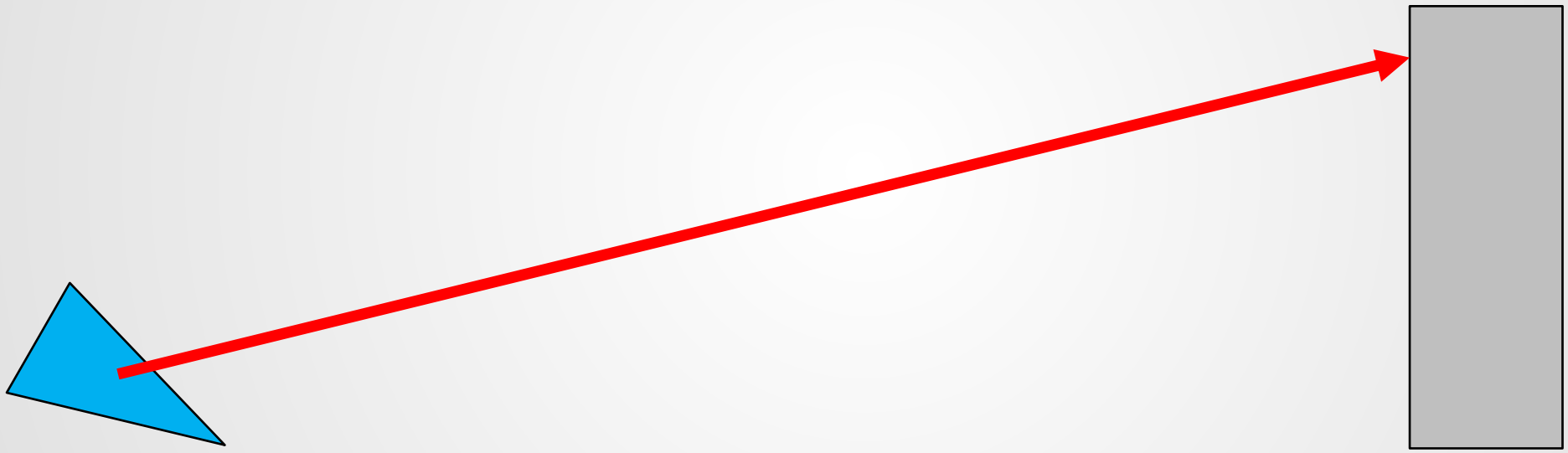
- Using our robot example from before, the state is

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & \theta_k \end{bmatrix}^\top$$

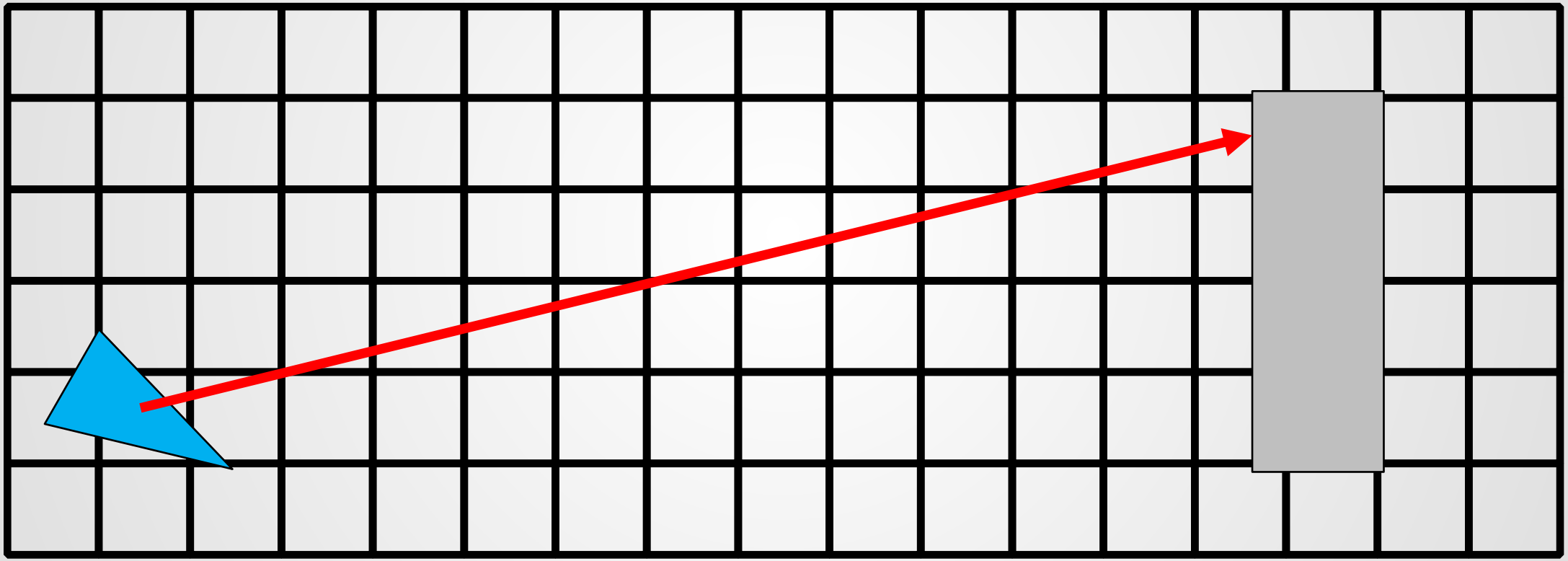
- The measurement is

$$\mathbf{z}_k^s = \begin{bmatrix} r_k^s \\ \beta_k^s \end{bmatrix}$$

Occupancy Grid Example



Occupancy Grid Example



Updating an Occupancy Grid

- **The real-valued coordinate of the return from the sensor is**

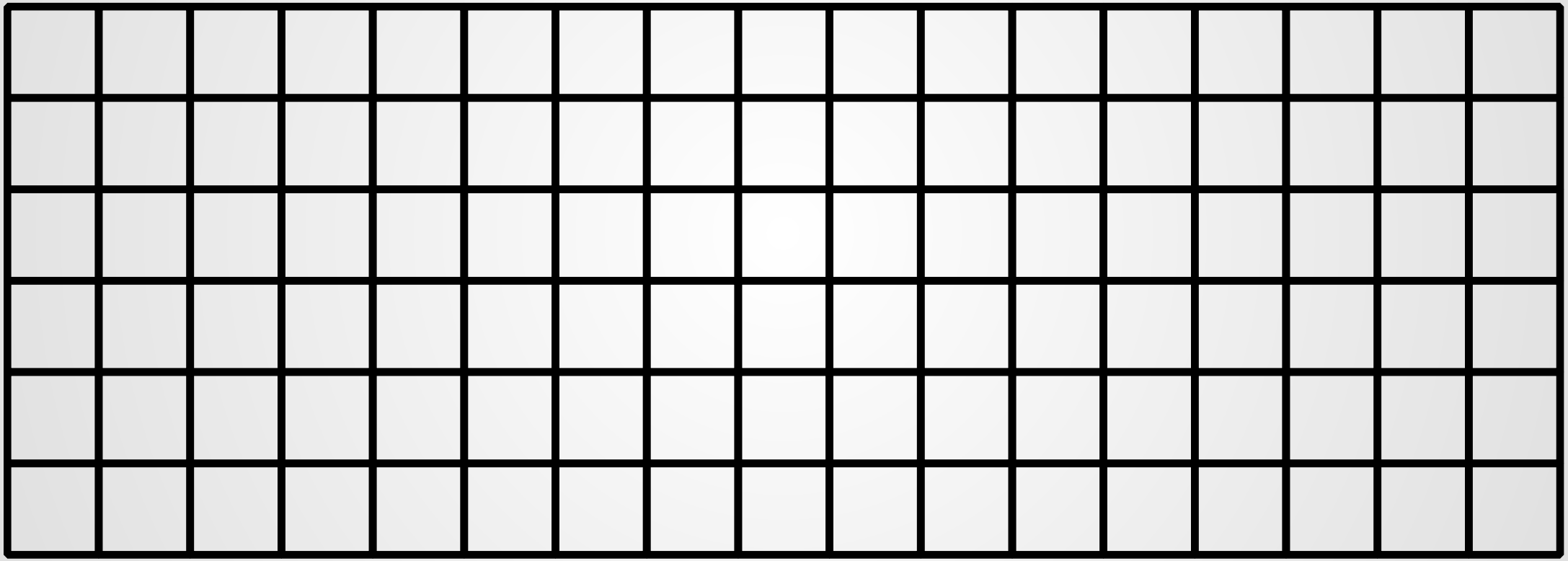
$$x_k^s = x_k + r_k^s \cos\left(\beta_k^s + \theta_k\right)$$
$$y_k^s = x_k + r_k^s \sin\left(\beta_k^s + \theta_k\right)$$

Updating an Occupancy Grid

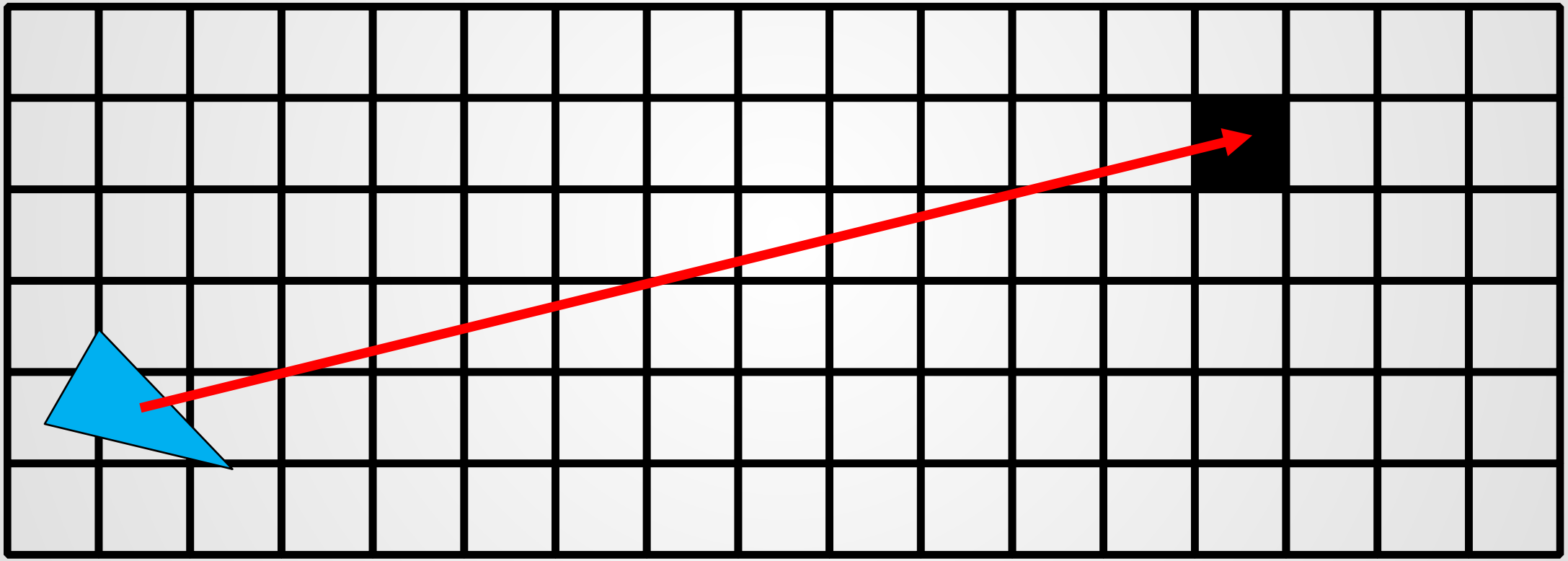
- **We convert these to grid coordinates by rounding,**

$$i = \lfloor x_k^s / c_w \rfloor$$
$$j = \lfloor y_k^s / c_h \rfloor$$

Initial Grid



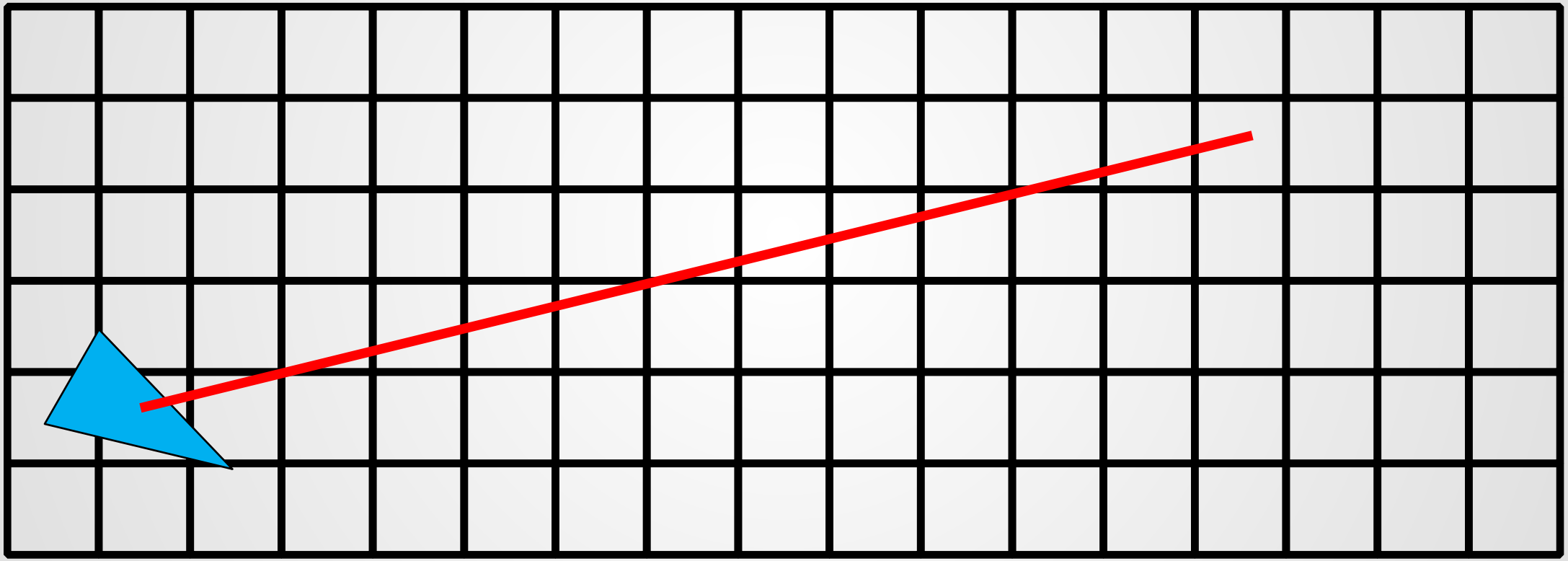
Occupancy Grid Example



Limited Sensor Range

- Most robot sensors have a maximum detection range
- If they are operating in a wide open area then the sensor will not return any range information
- In the basic occupancy grid, this means that we can’t perform any updates

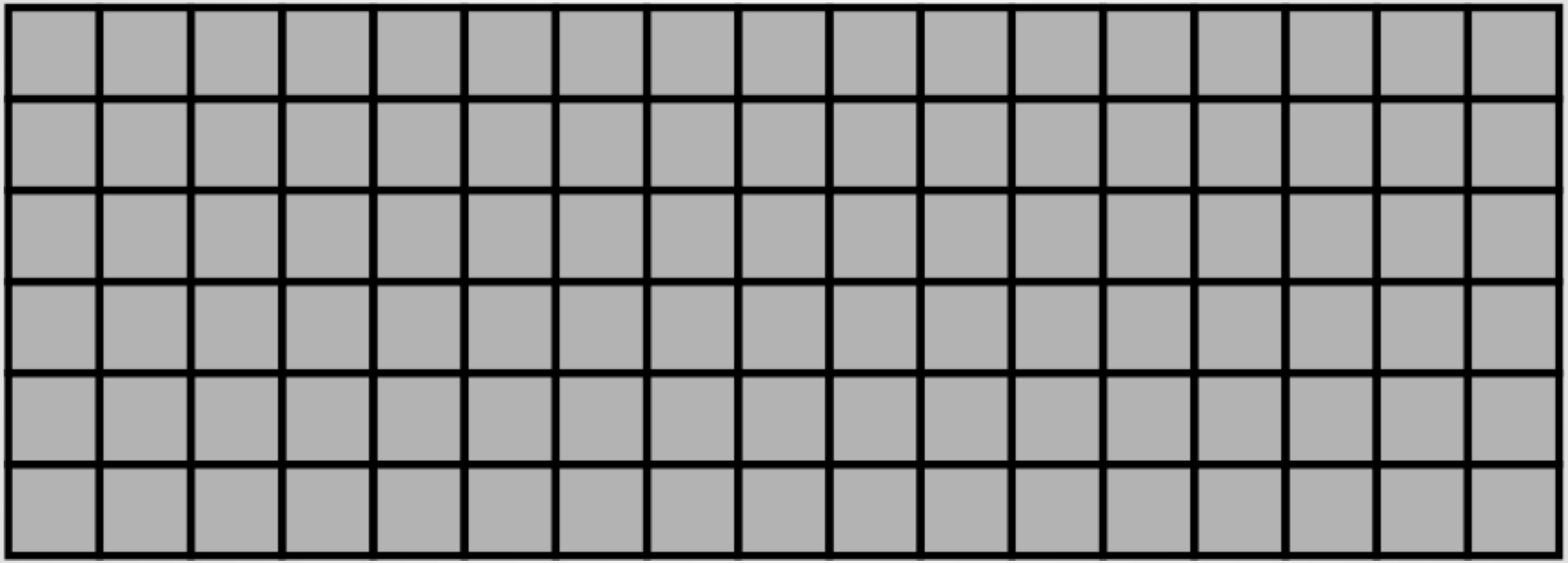
Occupancy Grid Example



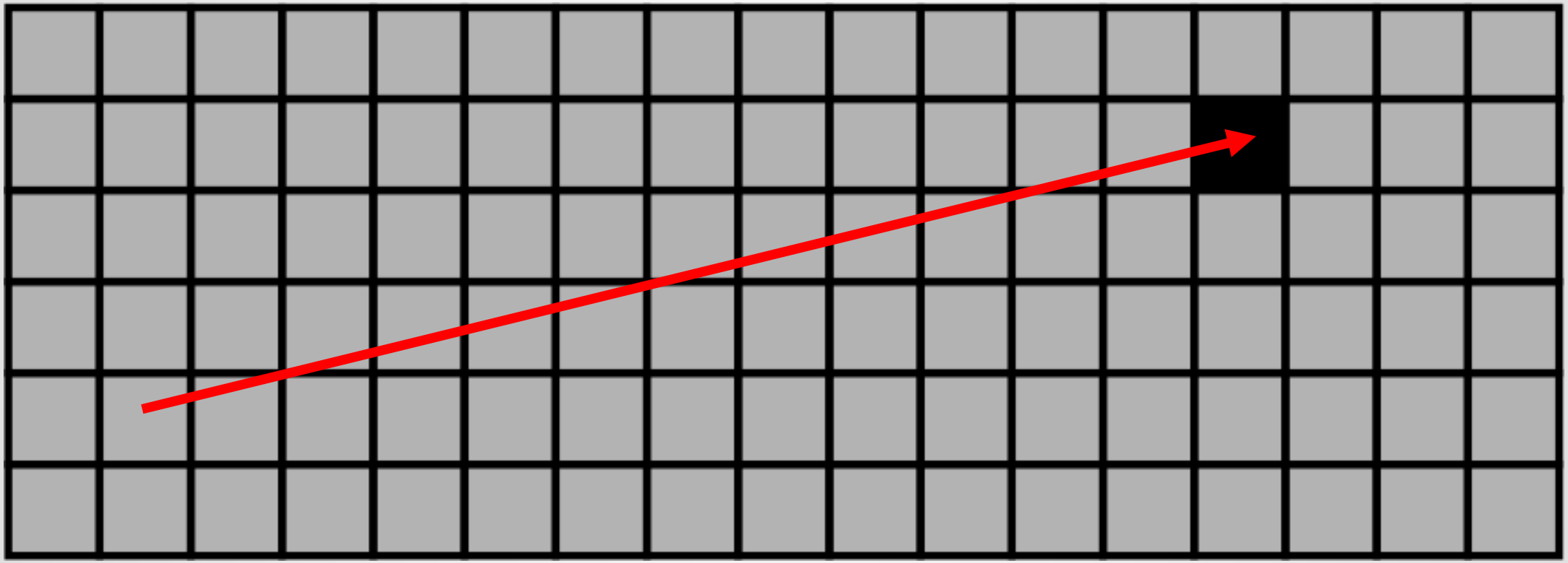
Inference Grids

- Binary values let us treat the grid as a spatially quantized version of a point cloud
  - We keep track of just where the occupied cells are
  - The cells naturally aggregate points for compression
- However, recall we said we wanted to use the grid to model free space that the robot can move in
- We can see this by introducing an *unknown cell state*
- These are cells which haven't been illuminated by the sensor yet

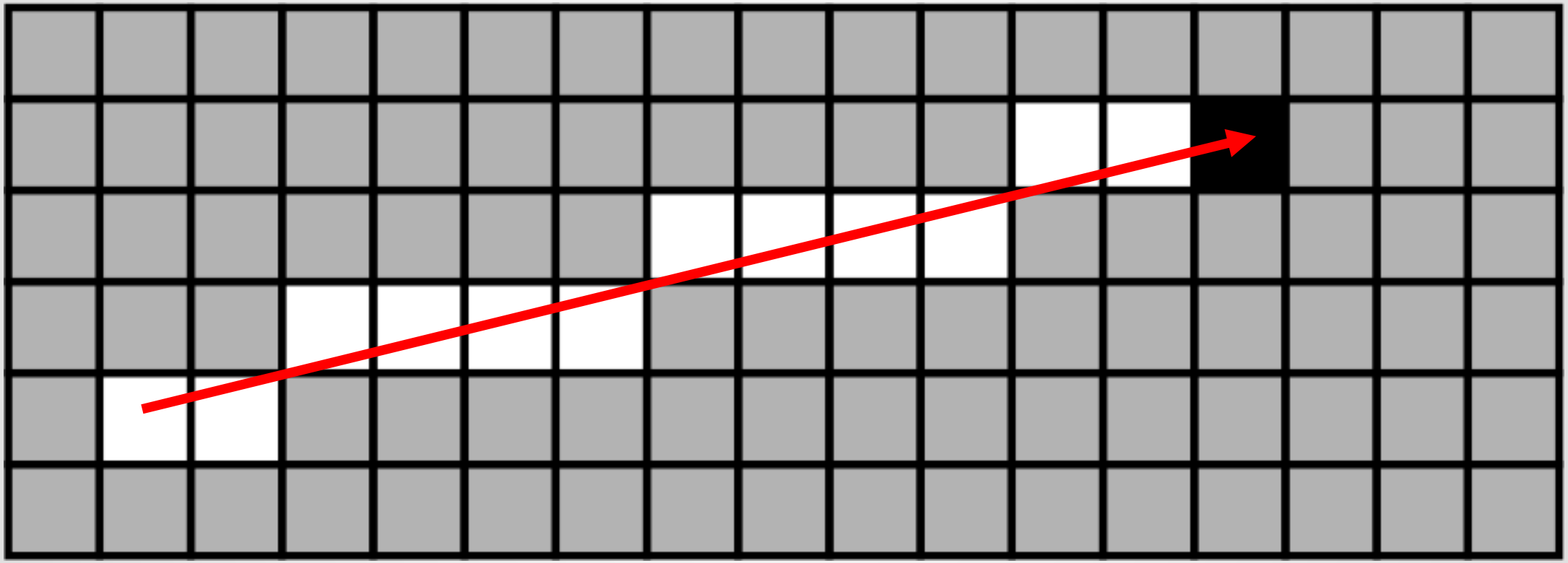
Initial Map with Uncertain Occupancy State



Updating the Grid



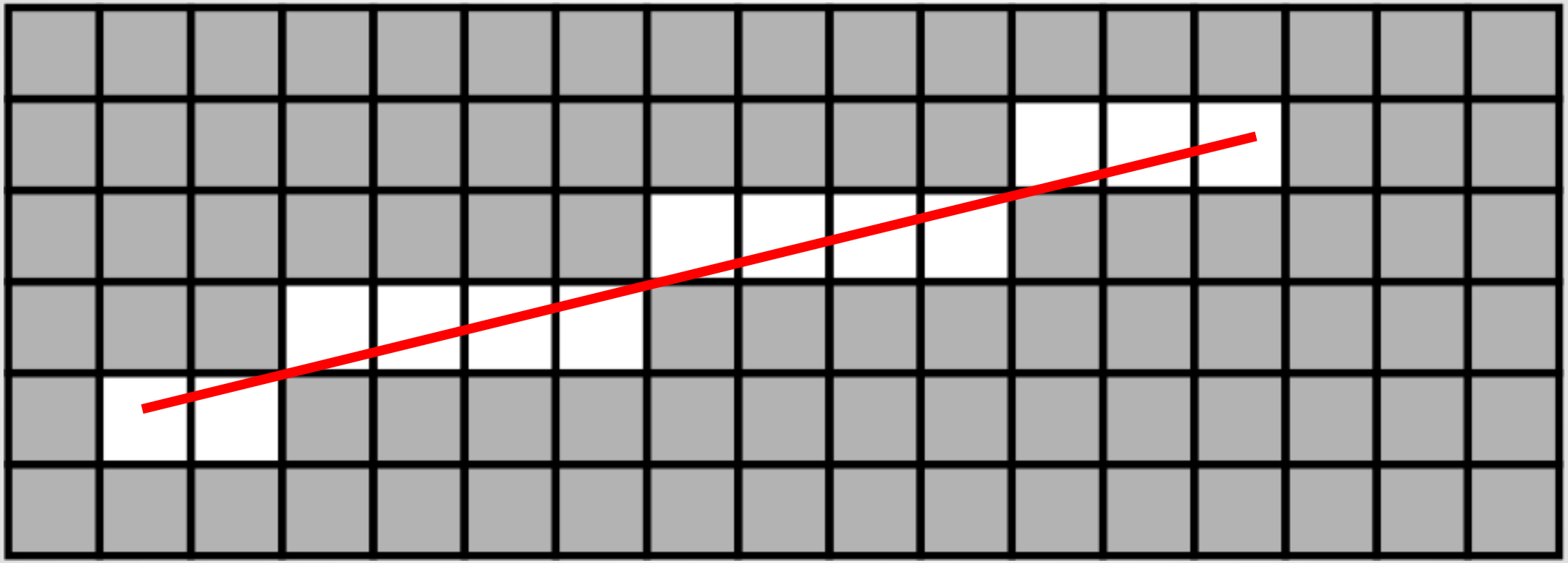
Updating the Grid



Limited Sensor Range

- Even if the sensor doesn’t detect a hit it tells us that the robot that, if there are any obstacles, they are beyond the sensor detection range
- This provides the robot with information to, for example, let it know that there are areas of the map which have not been explored yet

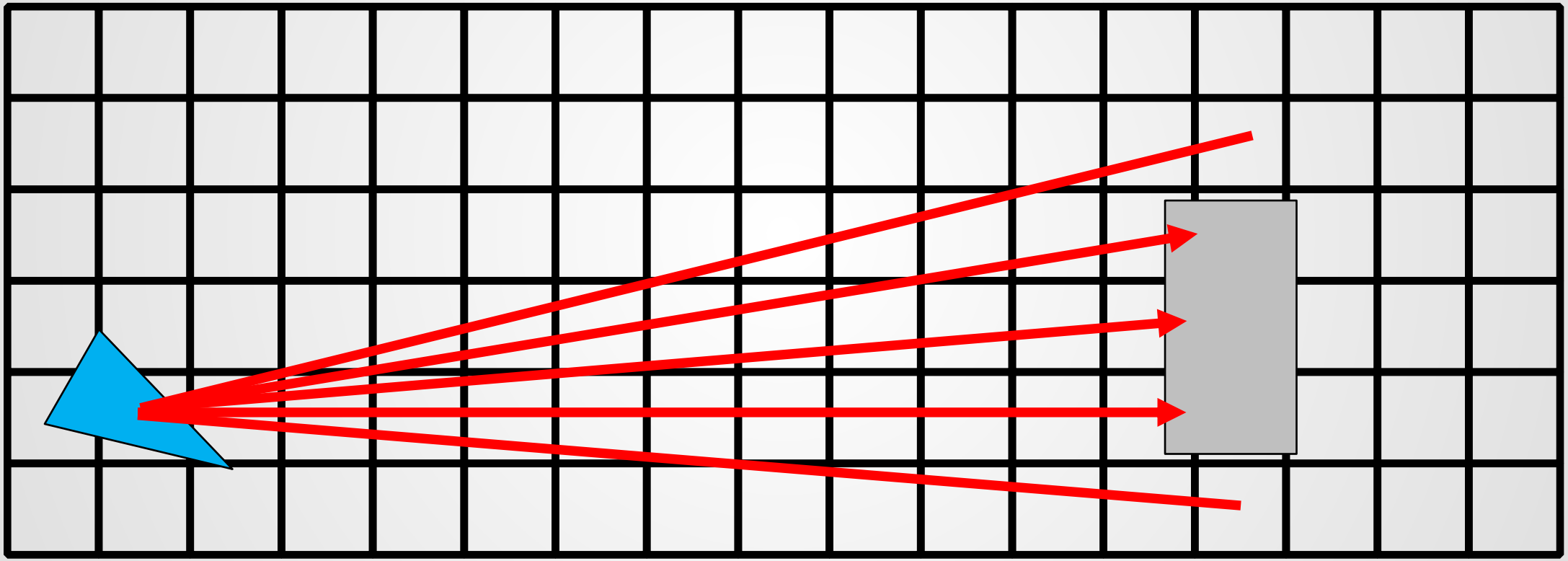
Updating the Grid with No Detections



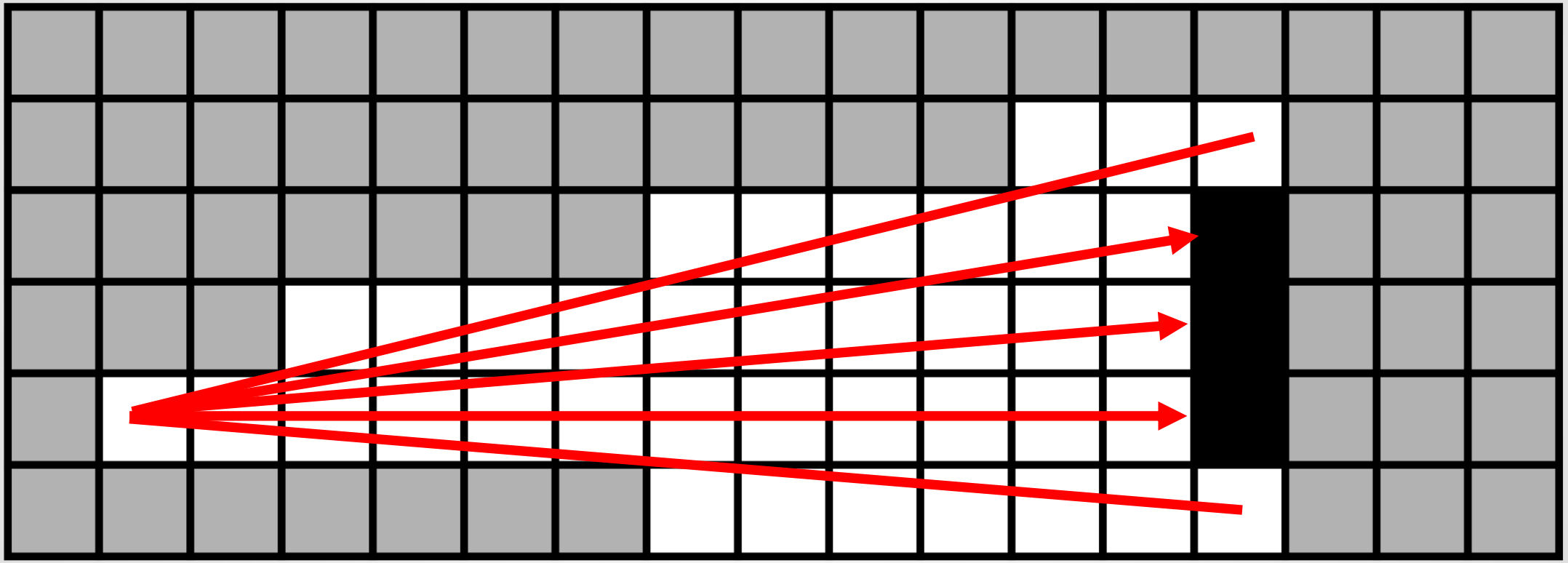
Updating with Multiple Rays

- When we have multiple rays, we simply superimpose them on the grid and take the union of the results

Updated Grid with Multiple Rays



Updated Grid with Multiple Rays



Updating with Multiple Platform Positions

- We project the rays we get at each position and take the union of the results

Perfect is *Easy*

- So far, we've assumed that everything is perfect:
  - There are no false negatives
  - There are no false positives
  - There is no measurement noise
  - The measurement range is infinite
- In the perfect case, updating the grid is really simple:
  - For each ray, you construct the list of cells the ray contacts
  - For every cell which the ray passes through you mark as unoccupied
  - For a cell in which the ray detects an obstruction, mark as occupied

Occupancy Grids and Noisy Sensor Models

- However, in practice sensors are noisy and so the deterministic solution can't be directly applied
- Rather, we'll need to use a probabilistic approach to update the grid
- Before we carry out the update, we need to ask what is the probabilistic version of the occupancy grid

Probabilistic Occupancy Grids

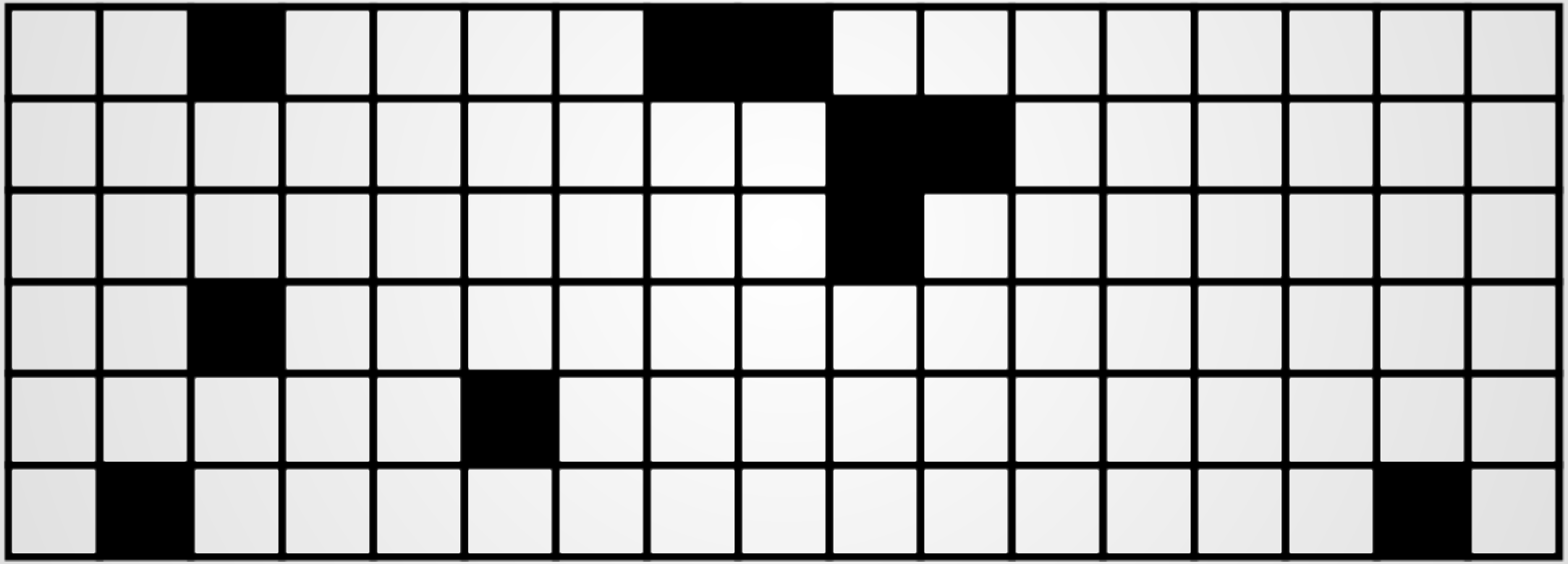
- *Deterministic Occupancy Grids*
- *Updating Deterministic Occupancy Grids in the Ideal Case*
- Probabilistic Occupancy Grids
- *Updating Probabilistic Occupancy Grids with Noisy Sensors*
- *Scan Matching, Odometry and Occupancy Grids*
- *Particle Filters and FastSLAM*
- *Suboptimal Particle and Factor Graph Approaches*

Probabilistic Occupancy Grids

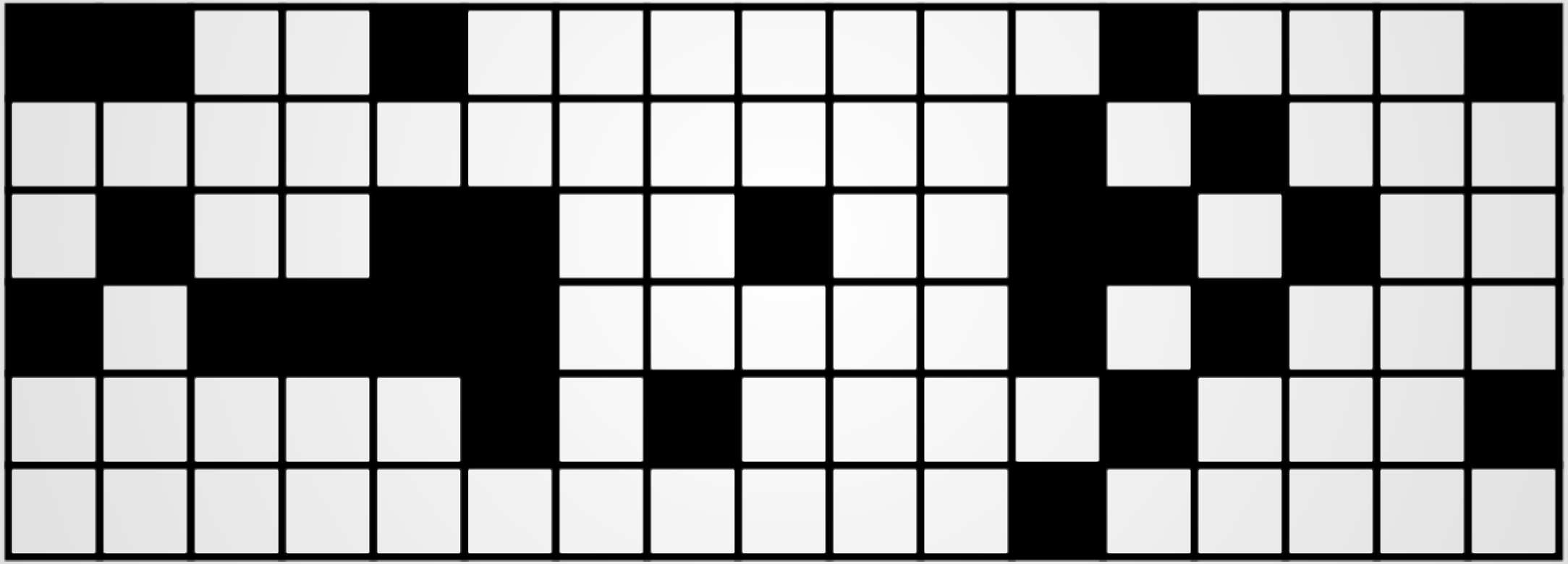
- **The errors in the sensor models introduce uncertainty**
- **We want to describe and model it *probabilistically***
- **Therefore, we define a probability distribution *over the occupancy grids*,**

$$\Pr(O)$$

Occupancy Grid Example



Occupancy Grid Example



Probabilistic Occupancy Grids

- **The probability distribution specifies what's the probability that the *entire* occupancy grid takes a particular configuration,**

$$\begin{aligned}\Pr(O) &= \Pr(O = G) \\ &= \Pr(S(C_1) = S(G_1) \wedge \cdots \wedge S(C_N) = S(G_N))\end{aligned}$$

Probabilistic Occupancy Grids

- To compute this, we could assume that the probability of every cell is independent of every other cell,
- Therefore, the probability would be given by

$$\Pr(O) = \Pr(O = G)$$

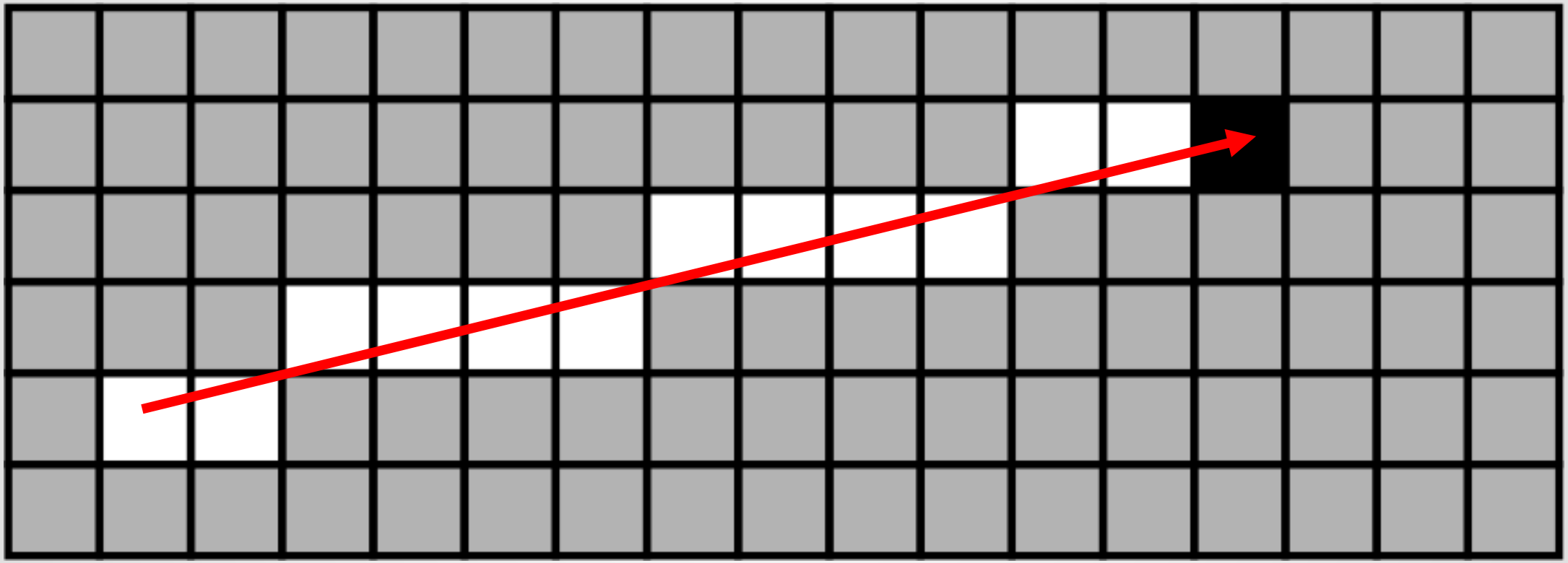
$$= \prod_{i=1}^{N} \Pr(S(C_i) = S(G_i))$$

Limitations with the Independence Approach

- However, the occupancy probabilities of each cell are *not* independent of one another

- The reason is because of the nature of the sensor:
  - A single measurement updates the states of multiple cells at the same time

Multiple Cells Affected per Observation



Full Approach

- **Therefore, we can only write the probability as**

$$\begin{aligned}\Pr(O) &= \Pr(O = G) \\ &= \Pr(S(C_1) = S(G_1) \wedge \cdots \wedge S(C_N) = S(G_N))\end{aligned}$$

- **There is no first order simplification we can use**

Occupancy Grid Space

- We can write this more compactly and say that any valid occupancy grid is a member of the occupancy grid space,

$$O \in \mathcal{O}$$

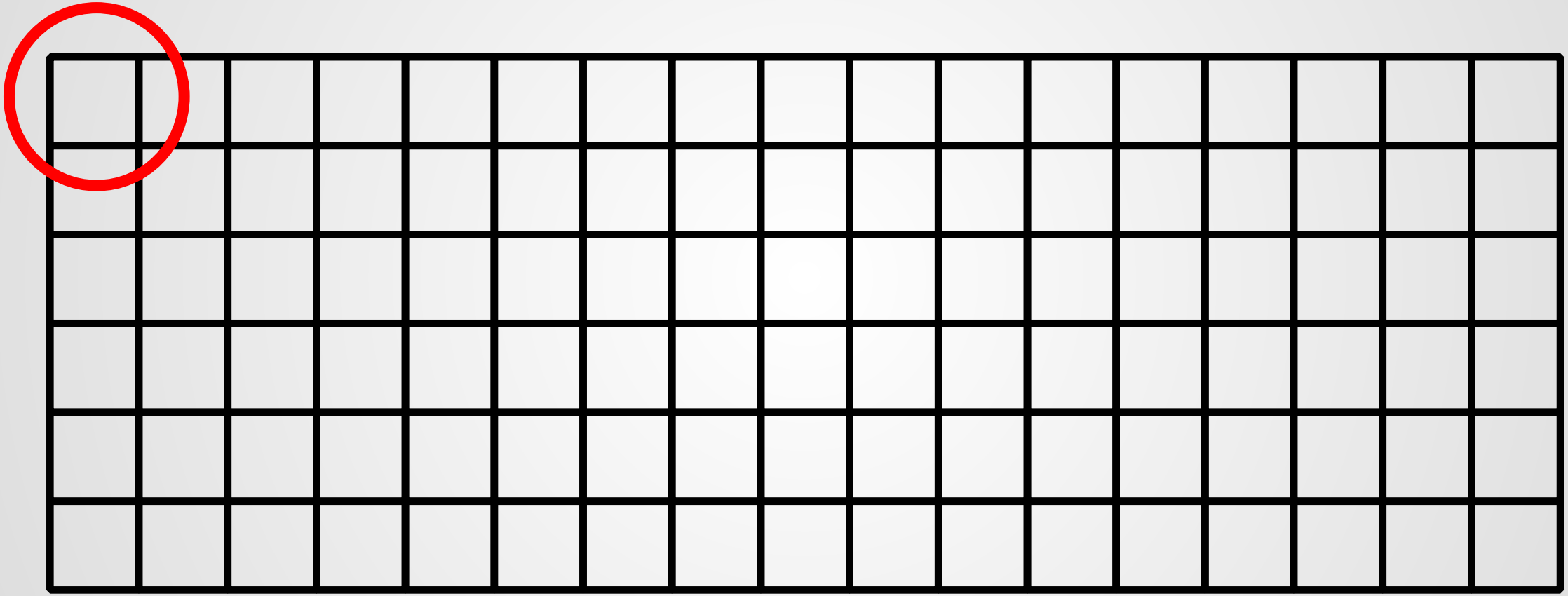
- The total number of possible occupancy maps is

$$|\mathcal{O}| = 2^N$$

Computing Probability of an Individual Cell

- Therefore, the probability distribution is defined over the entire grid
- However, we often care about the occupancy probability of a particular cell C_n
- We can work this out through *marginalization*

Computing Probability of an Individual Cell



Computing Probability of an Individual Cell

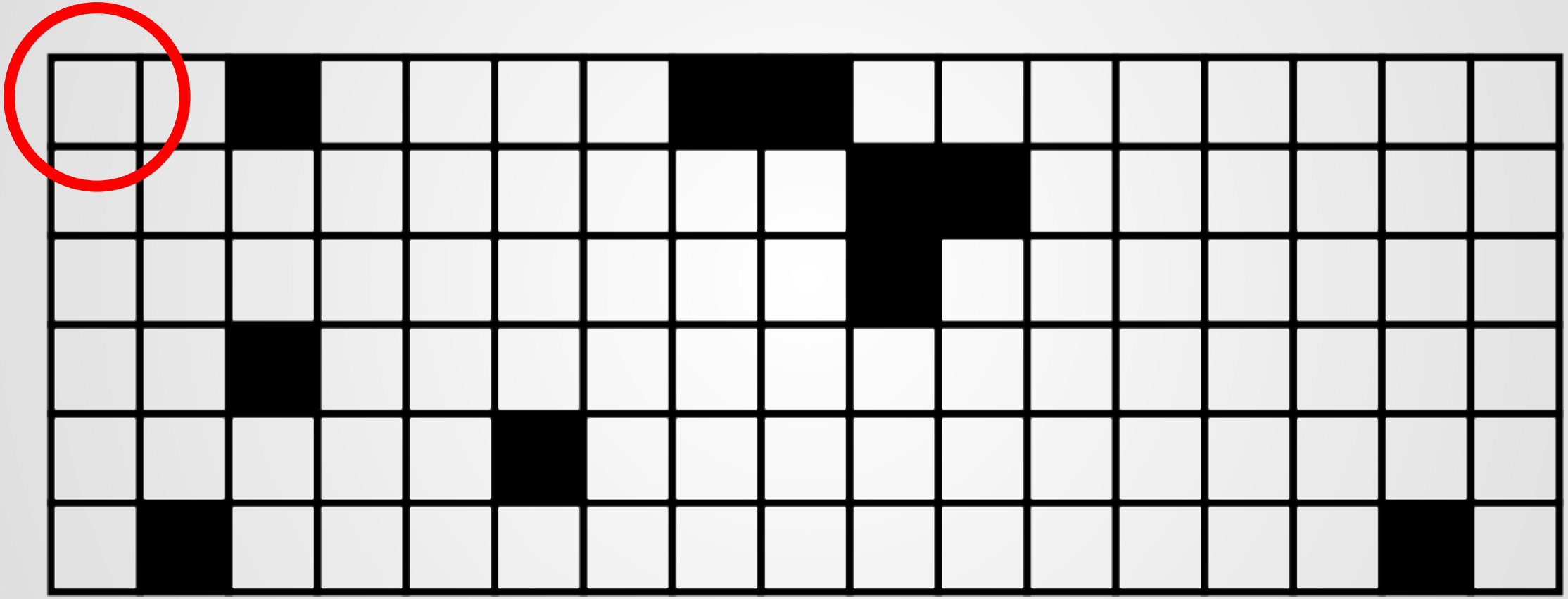
- We marginalize (=remove the effects of) all the other cells by summing up the probabilities from the *set* of all matching maps,

$$\Pr\left(S(C_n) = 0\right) = \sum_{G \in \mathcal{N}} \Pr\left(O = G\right)$$

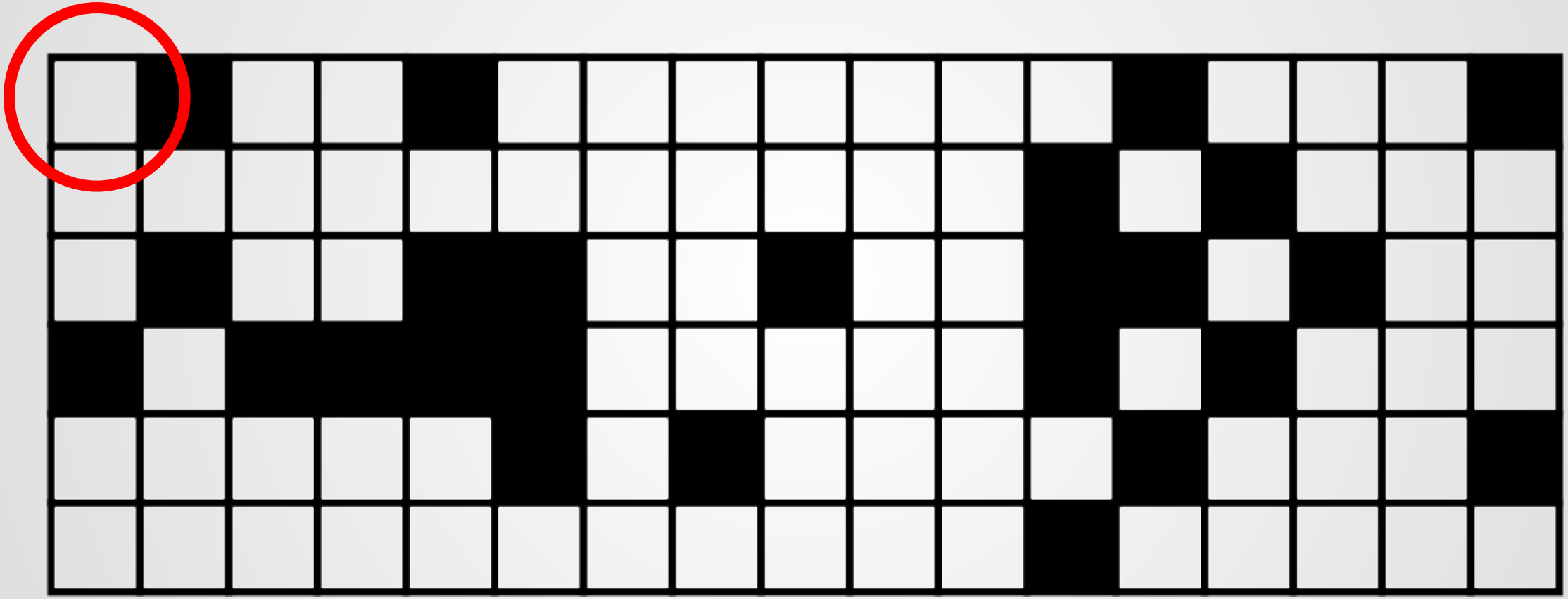
- The set $\mathcal{N}$ is the set of occupancy grids in which

$$S(C_n) = 0$$

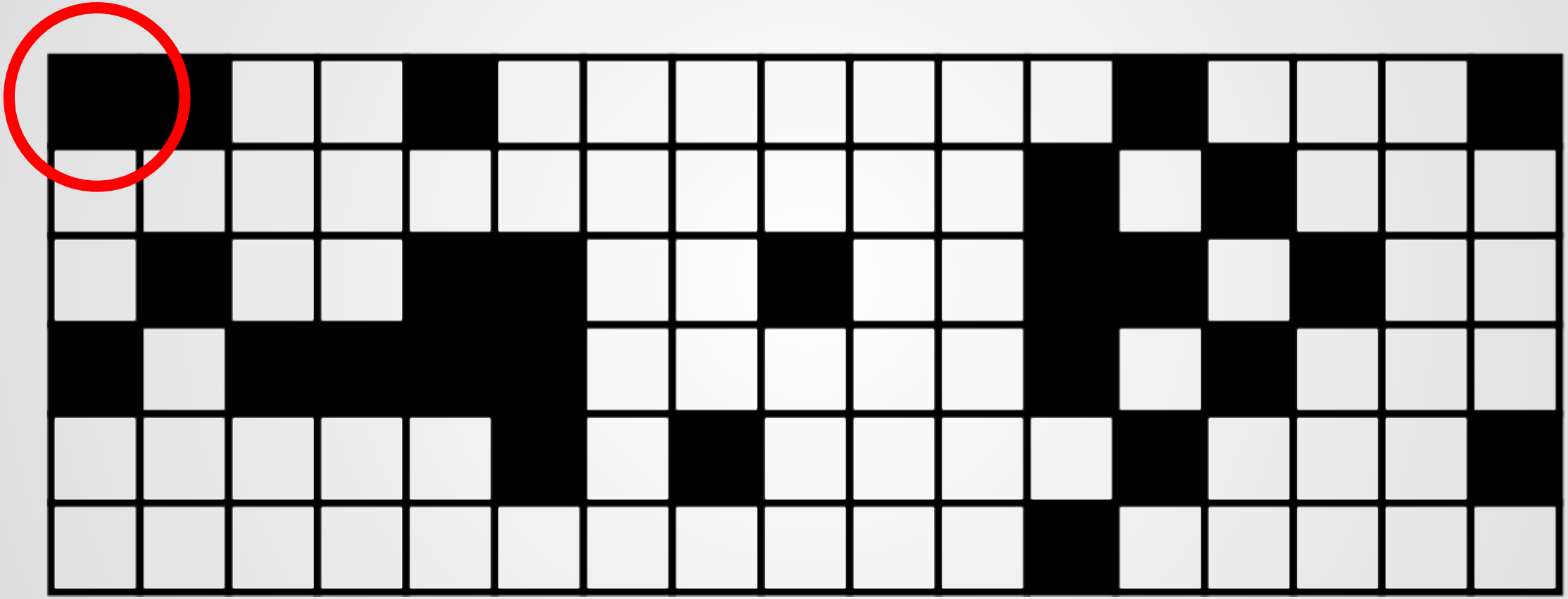
Computing Probability of an Individual Cell



Computing Probability of an Individual Cell



Occupancy Grid Example



Computing Probability of an Individual Cell

- **Since cells can only be occupied or unoccupied, it's the case that**

$$\Pr\left(S(C_n) = 1\right) = 1 - \Pr\left(S(C_n) = 0\right)$$

Mapping Cell Probabilities to Labels

- Recall for the deterministic case we said that cells could be unoccupied, occupied or unknown
- We can still label the cells based on the marginal probability

$$L\left(C_n\right) = \begin{cases} \text{free} & \text{if } \Pr\left(S(C_n) = 1\right) < \alpha \\ \text{unknown} & \text{if } \alpha \leq \Pr\left(S(C_n) = 1\right) \leq \beta \\ \text{occupied} & \text{if } \Pr\left(S(C_n) = 1\right) > \beta \end{cases}$$

- Typical values are $\alpha = 0.3, \beta = 0.7$

Updating Probabilistic Occupancy Grids

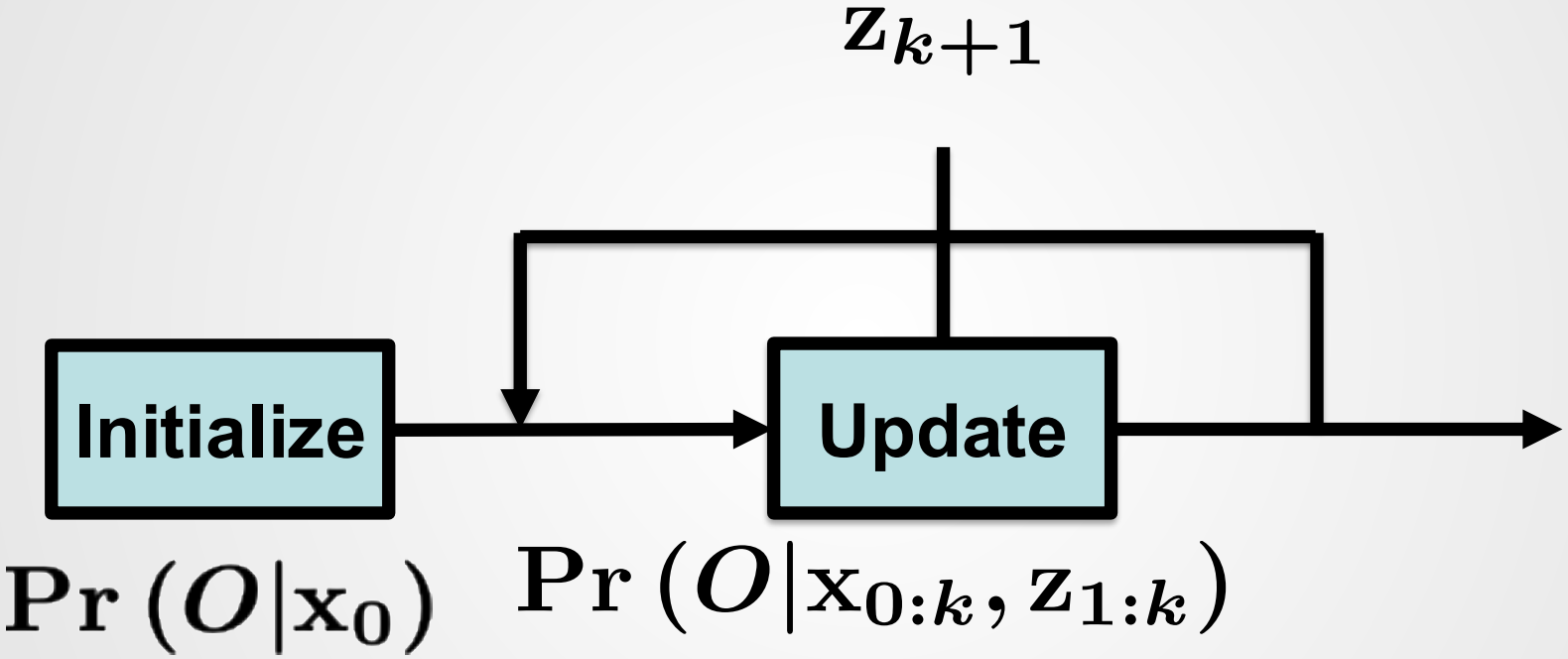
- *Deterministic Occupancy Grids*
- *Updating Deterministic Occupancy Grids in the Ideal Case*
- *Probabilistic Occupancy Grids*
- **Updating Probabilistic Occupancy Grids**
- *Scan Matching, Odometry and Occupancy Grids*
- *Particle Filters and FastSLAM*
- *Suboptimal Particle and Factor Graph Approaches*

Probabilistic Occupancy Grid Mapping

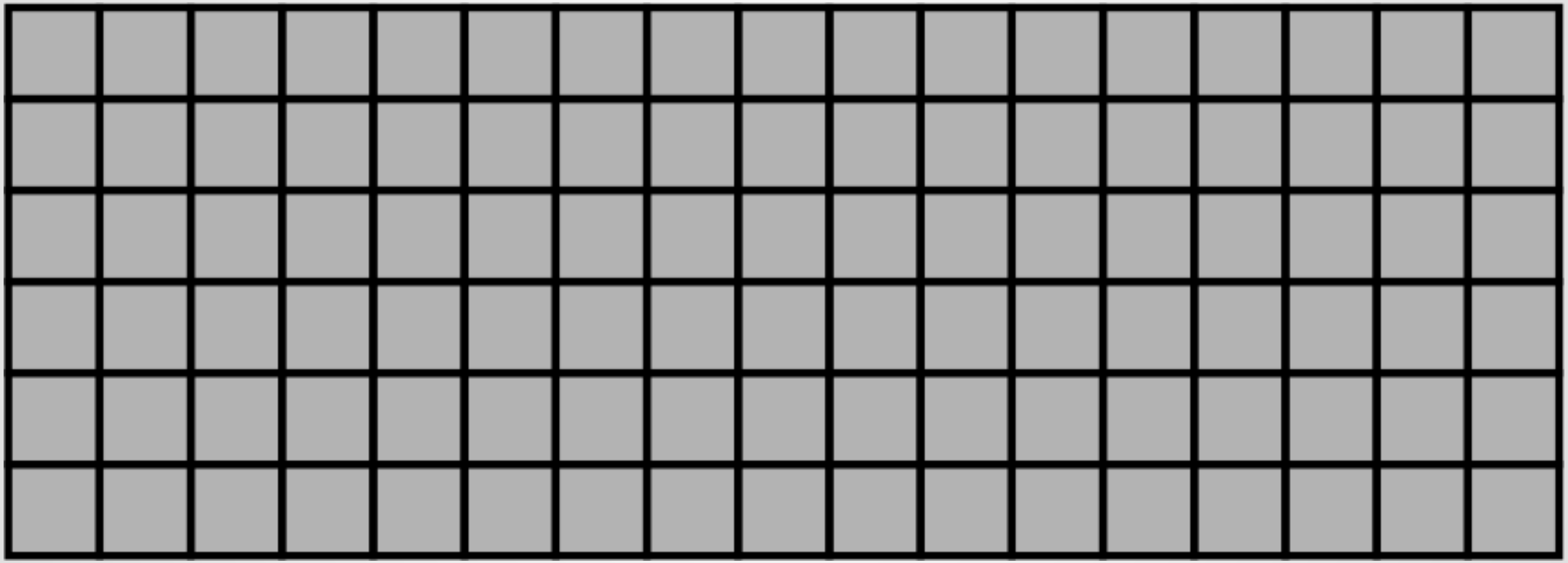
- We now want to sequentially estimate our map over time
- Recall that we assume that we know the position of the robot
- Therefore ,we want to compute the filtered solution

$$\Pr\left(O \middle| \mathbf{x}_{0:k}, \mathbf{z}_{1:k}\right)$$

Probabilistic Occupancy Grid Mapping



Initialization



Initialization

- We first have to initialize the map,

$$\Pr\left(O|\mathbf{x}_0\right)$$

- Since we don't know what the map looks like, a common assumption is that the occupancy probability of each cell is 0.5

$$\Pr\left(S(C_n) = 0|\mathbf{x}_0\right) = \Pr\left(S(C_n) = 1|\mathbf{x}_0\right) = 0.5$$

Initialization

- **This means that the probability of *any* grid is**

$$\Pr(O) = (0.5)^N$$

Updating the Grid

- **We update the grid using Bayes' Rule,**

$$\Pr\left(O|\mathbf{x}_1, \mathbf{z}_1\right) = \frac{\Pr\left(\mathbf{z}_1|O, \mathbf{x}_1\right)\Pr\left(O|\mathbf{x}_0\right)}{\Pr\left(\mathbf{z}_1|\mathbf{x}_{0:1}\right)}$$

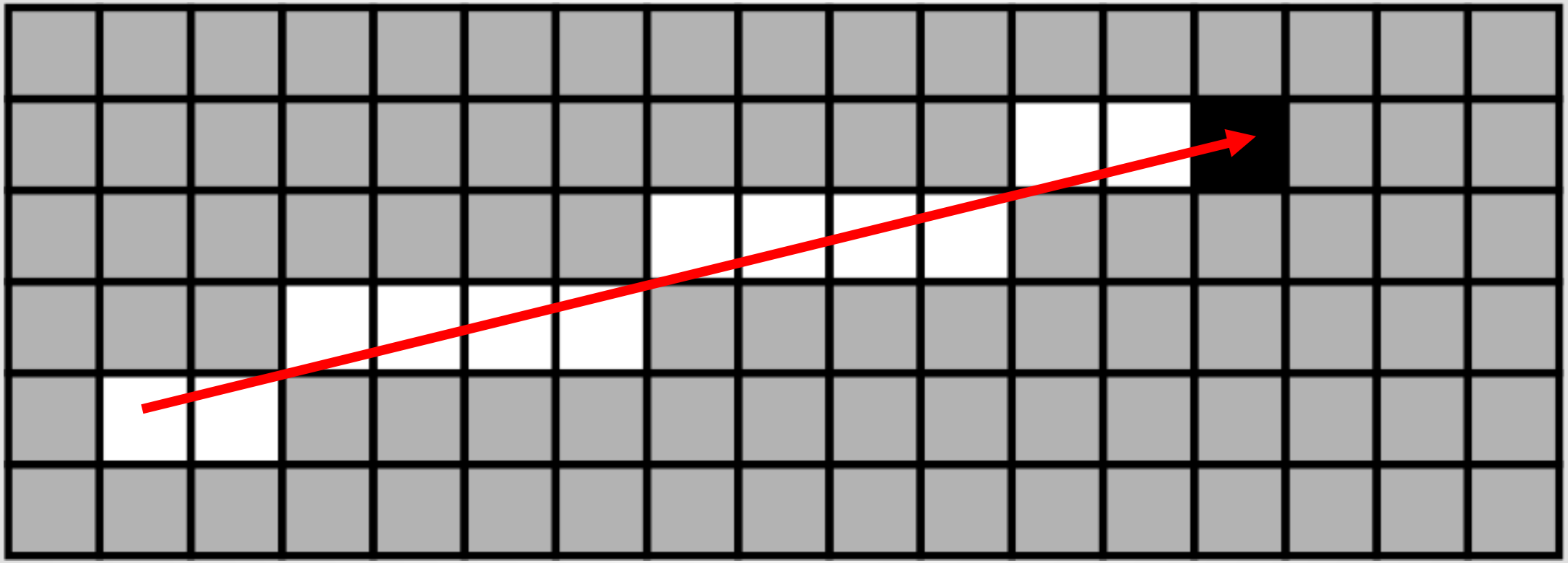
Updating the Grid

- To compute the update, we need to know the sensor likelihood

$$\Pr\left(\mathbf{z}_1 | O, \mathbf{x}_1\right)$$

- We have to look at our figure involving the sensor data again
- Let's start by considering the case where there's no noise in the sensor

Updating the Grid



Updating the Grid

- **Consider the set of all occupancy grids which are *consistent* with the observation**

$$\mathcal{Z}_{k+1} = \{G : G \text{ cons}\left(\mathbf{z}_{k+1}\right)\}\}$$

Updating the Grid

- **We then construct the likelihood from**

$$\Pr\left(\mathbf{z}_1 | O, \mathbf{x}_1\right) = \begin{cases} 1 & \text{if } O \in \mathcal{Z}_{k+1} \\ 0 & \text{otherwise} \end{cases}$$

Updating the Grid

- **Recall we update the grid using Bayes' Rule,**

$$\Pr\left(O|\mathbf{x}_1, \mathbf{z}_1\right) = \frac{\Pr\left(\mathbf{z}_1|O, \mathbf{x}_1\right)\Pr\left(O|\mathbf{x}_0\right)}{\Pr\left(\mathbf{z}_1|\mathbf{x}_{0:1}\right)}$$

Probabilistic Formulation

- **The evidence is given by summing over all possible occupancy grids,**

$$\Pr\left(\mathbf{z}_1 | \mathbf{x}_{0:1}\right) = \sum_{O \in \mathcal{O}} \Pr\left(\mathbf{z}_1 | O, \mathbf{x}_1\right) \Pr\left(O | \mathbf{x}_0\right)$$

Probabilistic Formulation

- **Given the form of the likelihood, this term becomes [FIX]**

$$\Pr\left(\mathbf{z}_1 | \mathbf{x}_{0:1}\right) = \sum_{O \in \mathcal{X}} \Pr\left(\mathbf{z}_1 | O, \mathbf{x}_1\right) \Pr\left(O | \mathbf{x}_0\right)$$

Recursive Formulation

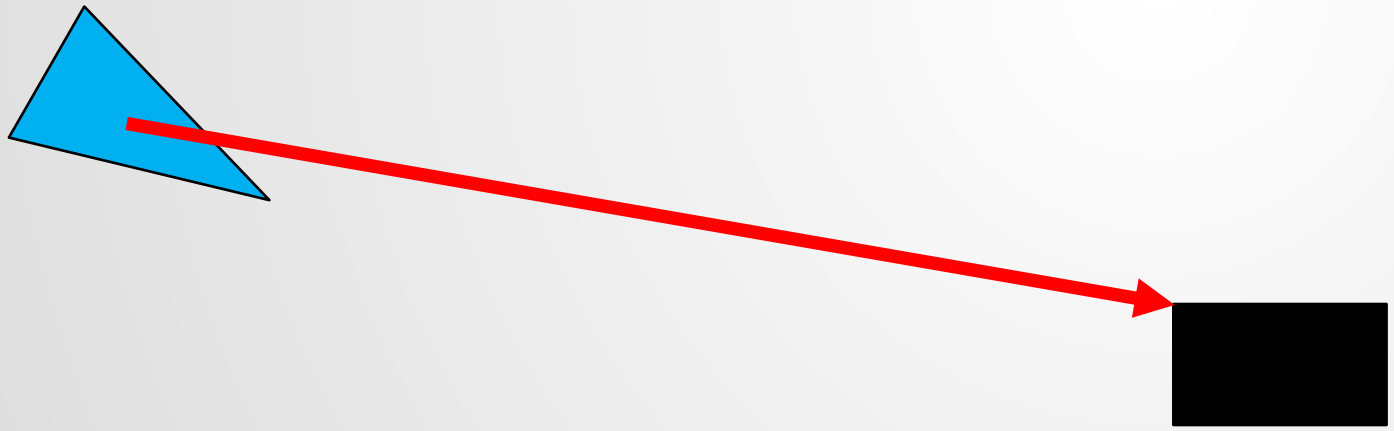
- We can now write this recursively as

$$\Pr\left(O \middle| \mathbf{x}_{0:k+1}, \mathbf{z}_{1:k+1}\right) = \frac{\Pr\left(\mathbf{z}_{k+1} \middle| O, \mathbf{x}_{k+1}\right) \Pr\left(O \middle| \mathbf{x}_{0:k}, \mathbf{z}_{1:k}\right)}{\Pr\left(\mathbf{z}_{k+1} \middle| \mathbf{x}_{0:k+1}, \mathbf{z}_{1:k+1}\right)}$$

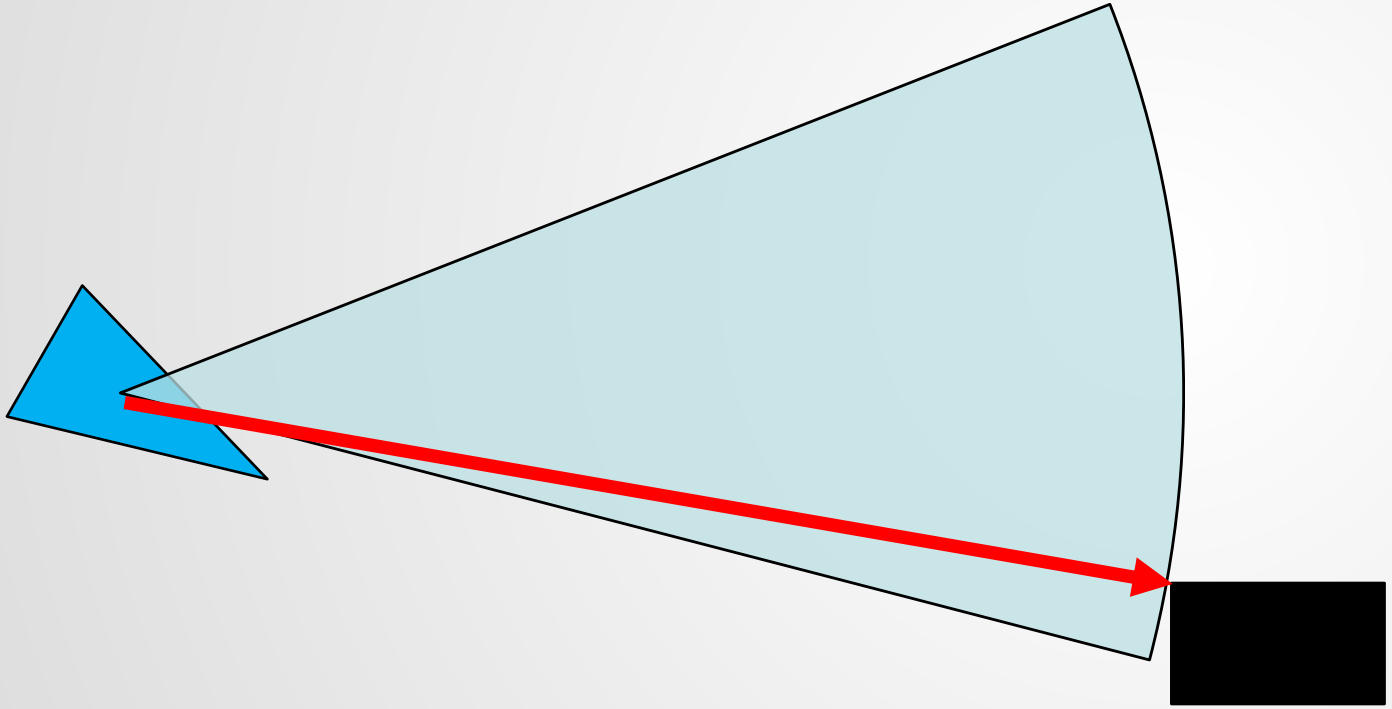
Wide Beam Width

- In the case with a perfect sensor, this might look like overkill
- However, it is used when some sensors (sonar, radar) can have quite a large beam width
- This means that when they detect the range to a surface, they mean that a target lies somewhere *along an arc*

Wide Beam Width



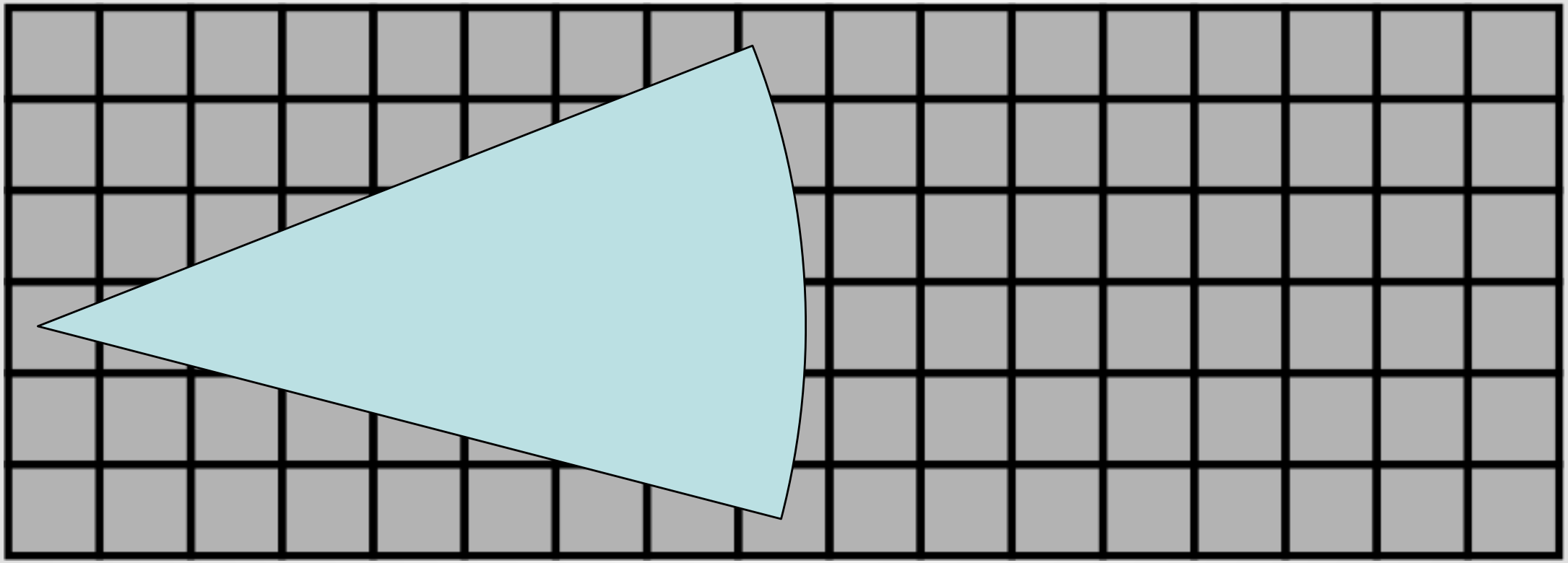
Wide Beam Width



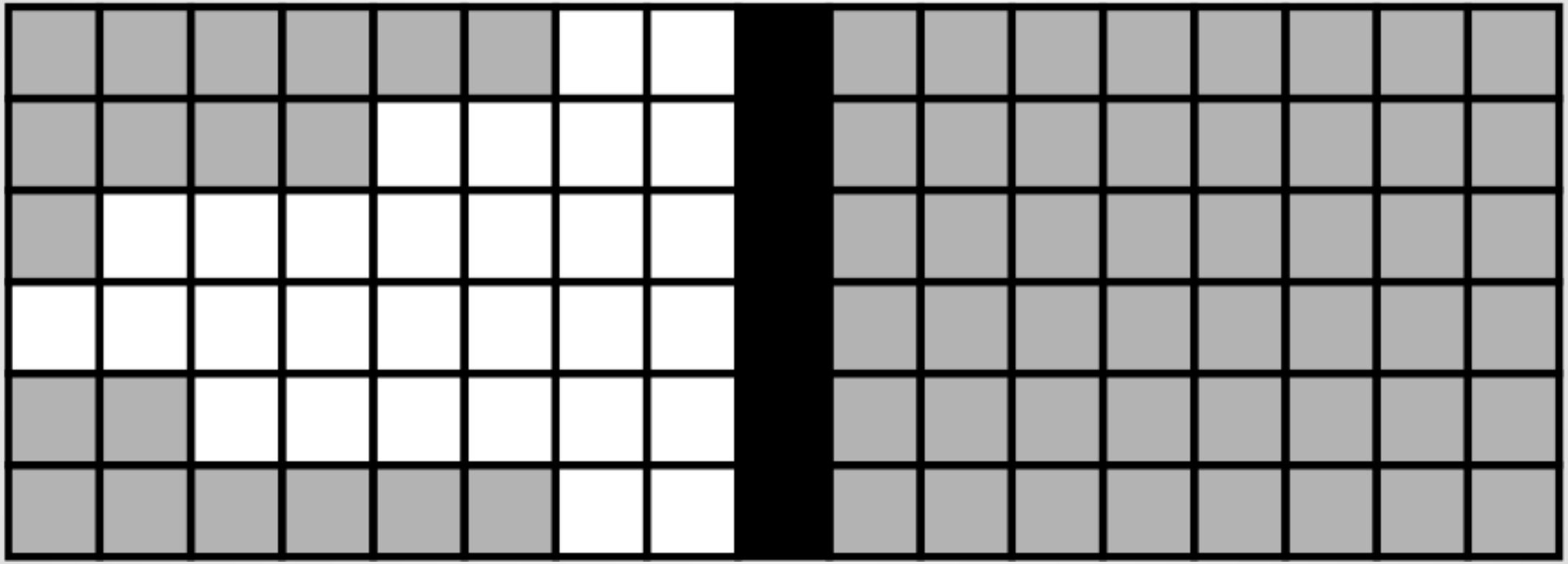
Wide Beam Width

- We might think that we can just model this return using the same structure we had with the narrow beam width case

Wide Beam Width



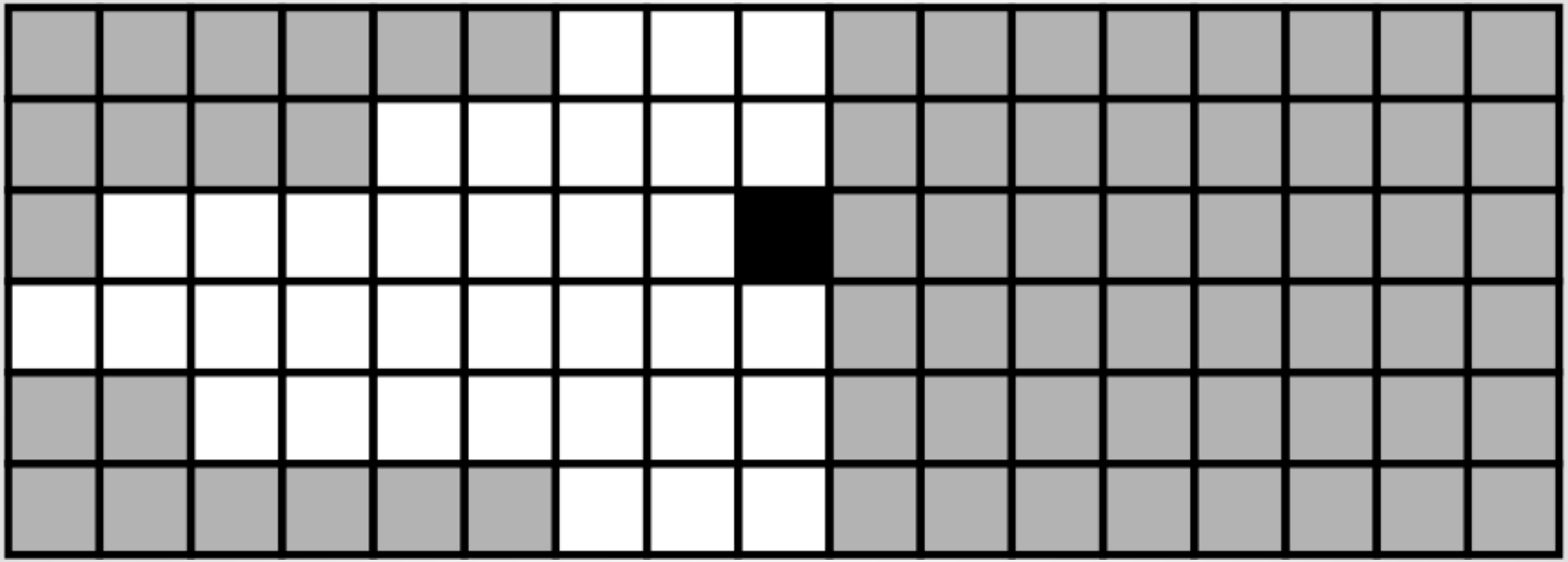
Wide Beam Width



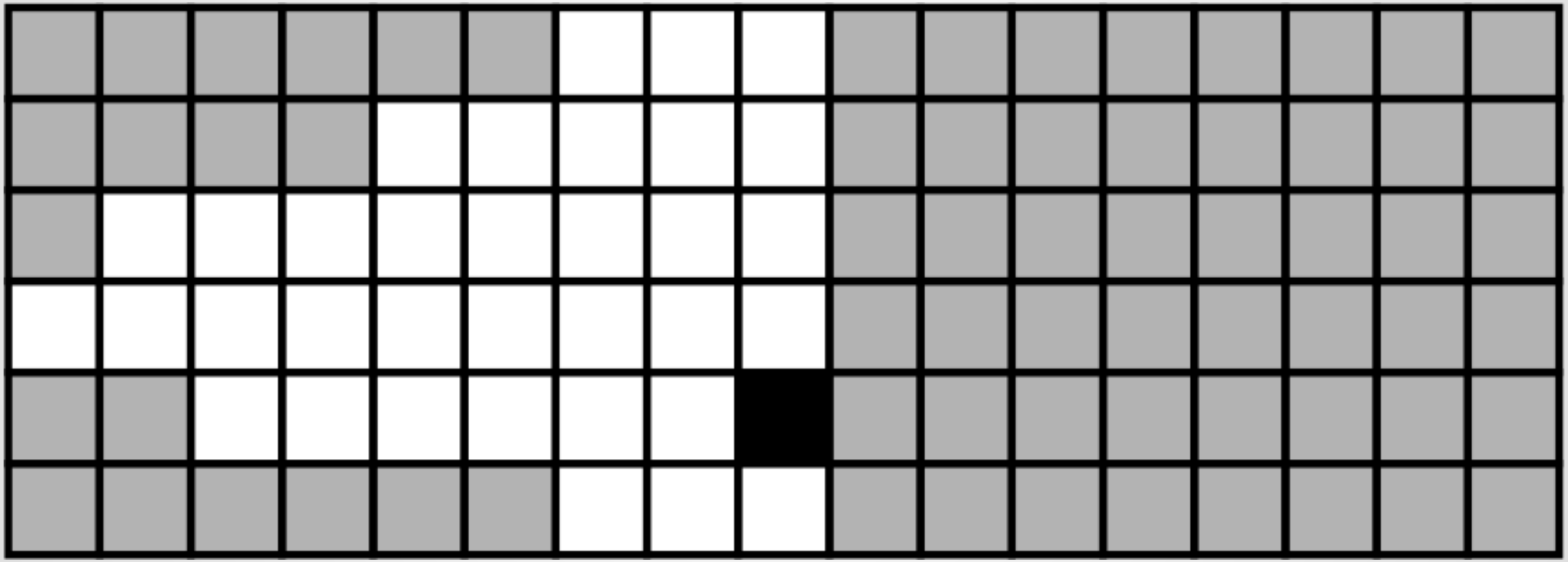
Wide Beam Width

- We might think that we can just model this return using the same structure we had with the narrow beam width case
- However, this assumes that the obstacle covers the *entire* arc of the sensor
- In actual fact, *any* combination of map which includes one or more cells on the beam arc is compatible with the observation

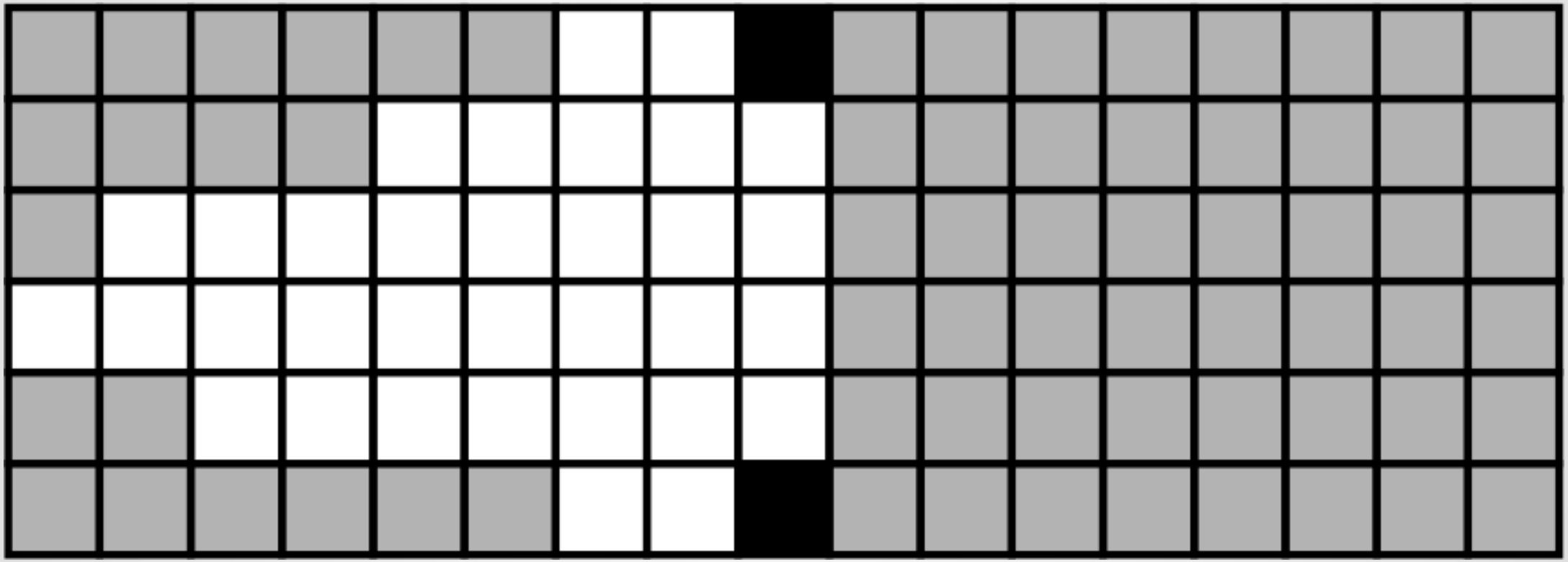
Wide Beam Width



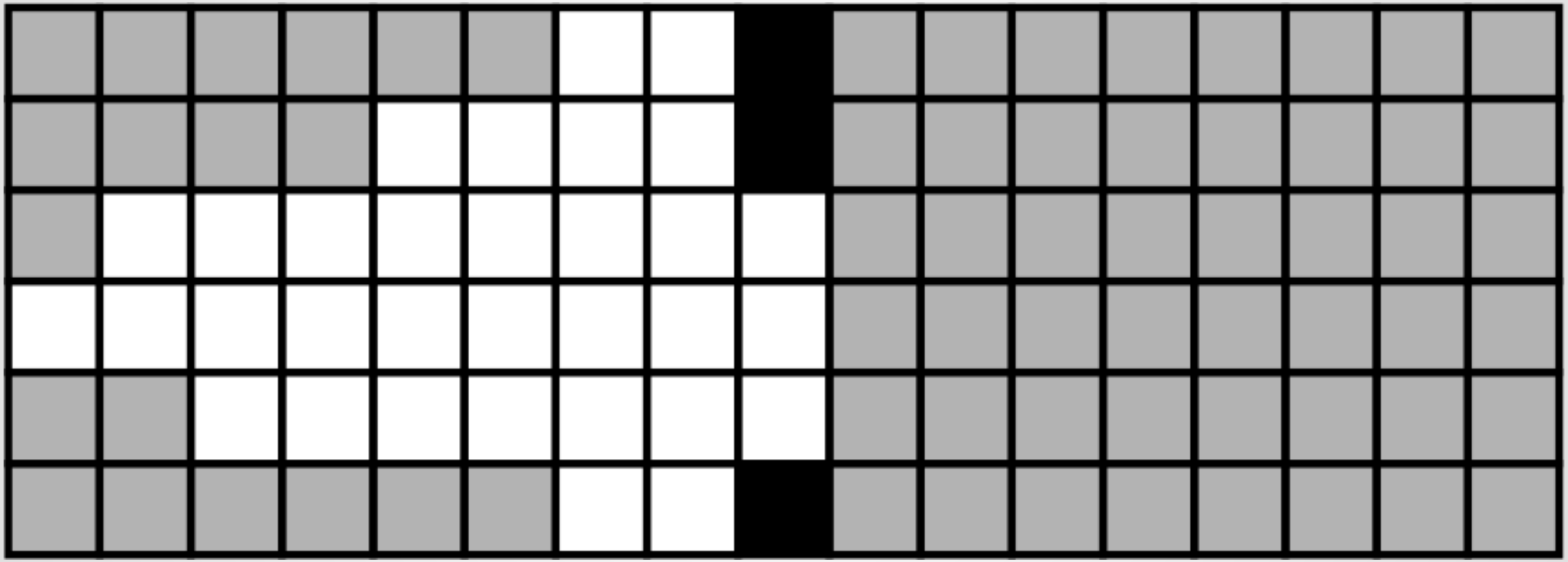
Wide Beam Width



Wide Beam Width



Wide Beam Width



Updating the Grid

- Again, we have to consider all grids consistent with the observation

$$\mathcal{Z}_{k+1} = \{G : G \text{ cons}\left(\mathbf{z}_{k+1}\right)\}\}$$

- We then construct the likelihood from

$$\Pr\left(\mathbf{z}_1 | O, \mathbf{x}_1\right) = \begin{cases} 1 & \text{if } O \in \mathcal{Z}_{k+1} \\ 0 & \text{otherwise} \end{cases}$$

Scalability

- However, the scalability of the full distribution is really bad
- For example, suppose we have an 8x8 grid
- This only has 64 cells
- The number of distinct occupancy grid configuratios that exists is

$$1.8447 \times 10^{19}$$

Scalability and Independence

- **Therefore, pretty much all occupancy grid SLAM systems assume that the occupancy probabilities are independent**

- **This will work okay if:**
  - **The angular errors *are small***
  - **The beam width of the sensor is less than a cell in size**

Scalability and Independence

- Using this simplification, the probability of a grid is just

$$\Pr(O = G) = \prod_{i=1}^{N} \Pr(S(C_i) = G_i)$$

- As we saw earlier, we just need to multiply the state of each cell together
- Also, we can just "read off" directly what the occupancy probability is

Updating with Independent Grid Cells

- **We can now update each cell independently using Bayes Rule,**

$$\Pr\left(S(C_i)|\mathbf{x}_{1:k}, \mathbf{z}_{1:k}\right) = \frac{\Pr\left(\mathbf{z}_k|S(C_i)\right)\Pr\left(S(C_i)|\mathbf{x}_{1:k-1}, \mathbf{z}_{1:k-1}\right)}{\Pr\left(\mathbf{z}_k\right)}$$

Sensor Noise

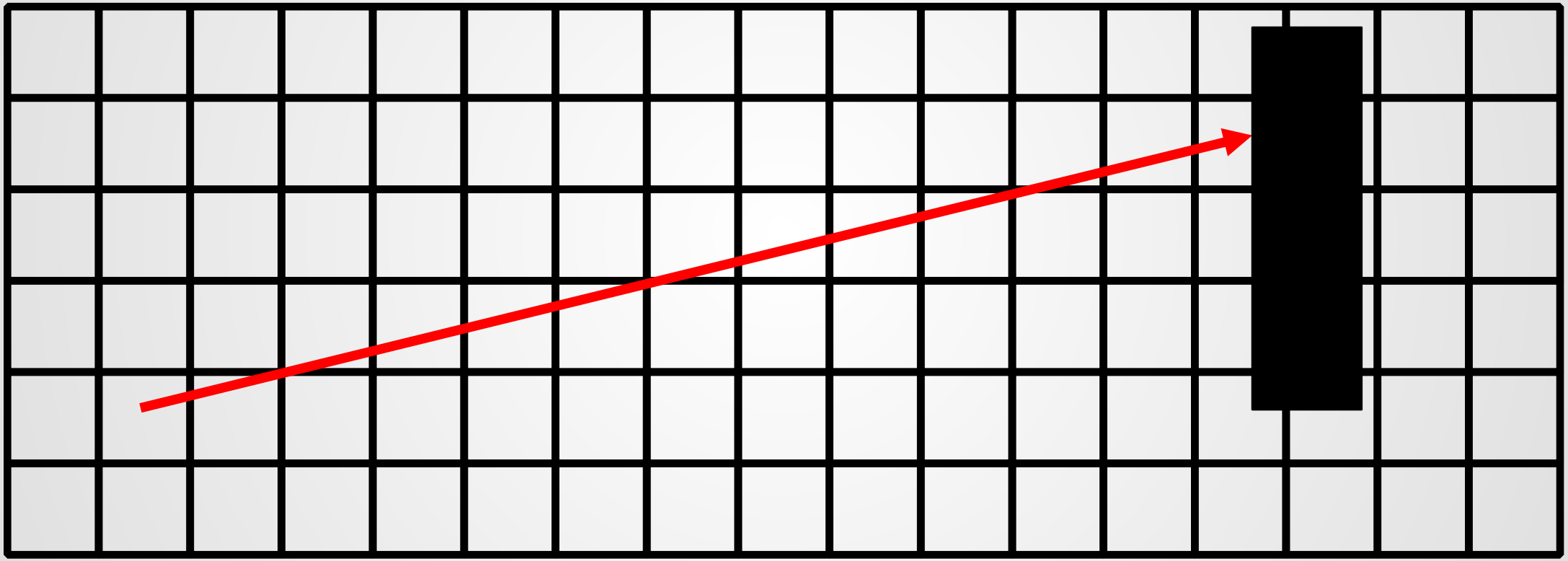
- We can now start adding sensor noise
- There are multiple types of sensor noise we can consider:
  - False detections
  - Missed detections
  - Range errors
  - Bearing errors

Range Errors

- Range errors mean that the sensor incorrectly measures the range to the surface
- For example, suppose the actual distance from the sensor to the closest surface is d
- The measured value could be distributed according to, say,

$$f\left(z_{k+1} | d_{k+1}\right) = \mathcal{G}\left(z; d_{k+1}, \sigma_R^2\right)$$

Range Errors



Range Errors

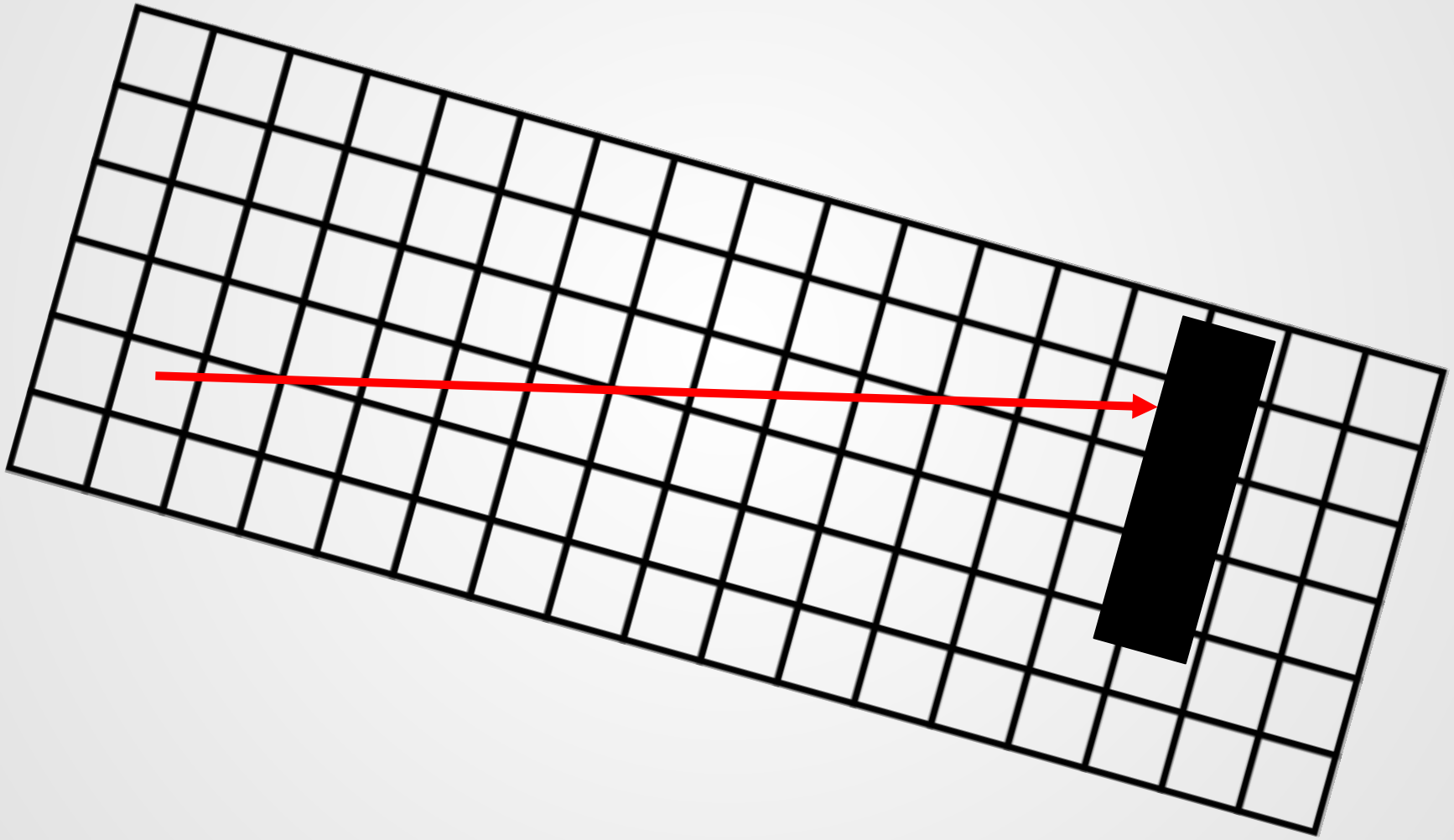
- When computing the update of a cell, we have to consider two things:
  - The cell the sensor is measuring
  - The occupancy state of the cell given that a ray has measured it

Cell Probabilities

- Given a Gaussian distribution, we assume we can define a range which contains the cell
- This produces a set of cells

$$\mathcal{R}$$

Range Errors

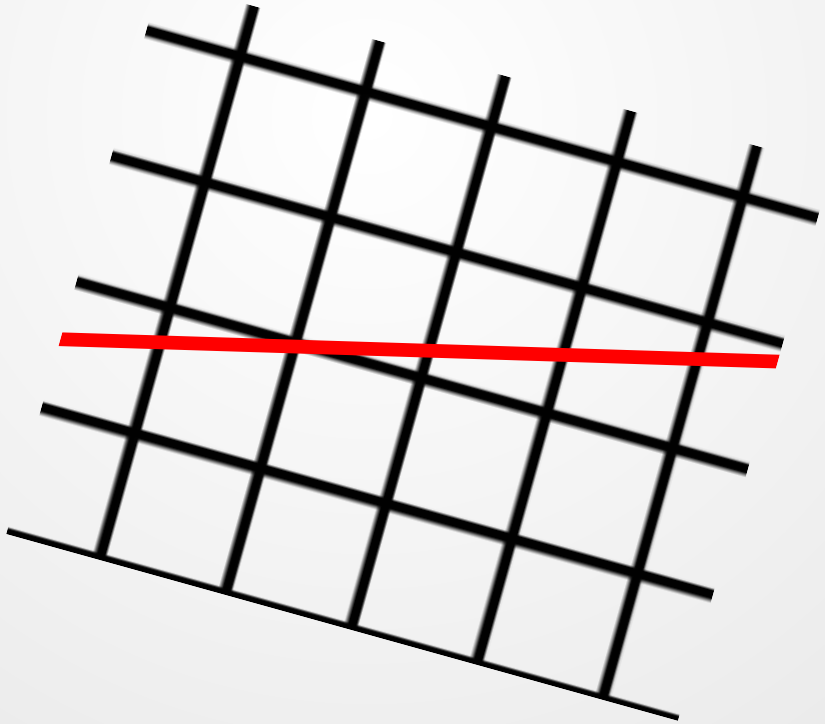


Cell Probabilities

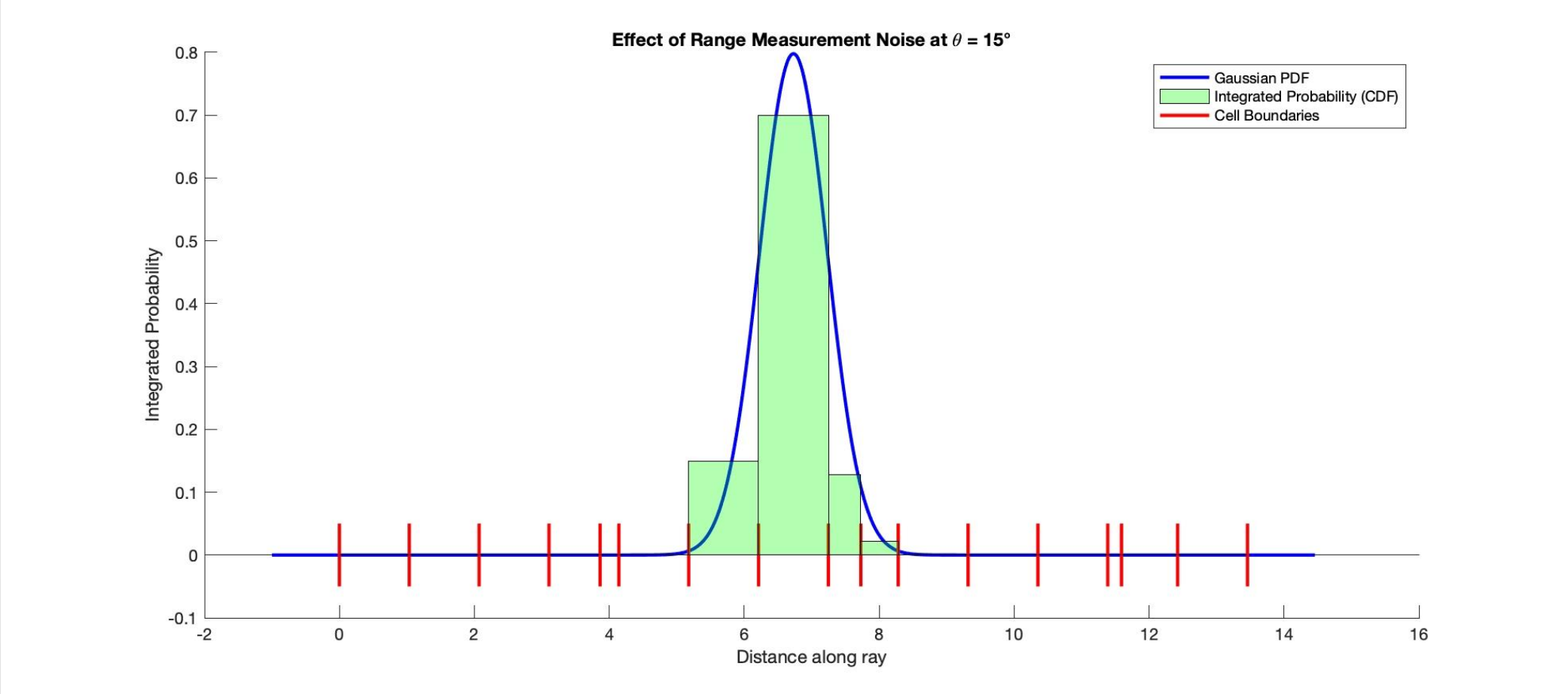
- To compute the probability of the ray ending up in the ray:
  - Identify the set of cells in $\mathcal{R}$ which need to be updated
  - For each cell work out the length of the ray which cuts through it
  - The probability for the cell is given by integrating the Gaussian over that cell

Range Errors

- For each cell in $\mathcal{R}$, compute the length of the ray passing through the cell



Measurement Likelihood



Log-Odds

- **Most implementations of occupancy grids use the log-odds formulation,**

$$\text{Odds}(x) = \frac{p(x)}{p(\neg x)} = \frac{p(x)}{1-p(x)}$$

$$l(x) = \log \frac{p(x)}{1-p(x)}$$

Algorithm Binary_Bayes_Filter(l_{t-1}, z_t)

1: $l_t = l_{t-1} + \log \frac{p(x|z_t)}{1-p(x|z_t)} - \log \frac{p(x)}{1-p(x)}$

2: **return** l_t

Log-Odds

- **The reason seems to be historic: in this form, Bayes' Rule can be written as addition rather than multiplication**

Algorithm Occupancy_Grid_Mapping($\{l_{t-1,i}\}$, x_t, z_t)

```
for all cells m_i do
    if m_i in perceptual field of z_t then
        l_{t,i} = l_{t-1,i} + Inverse_Sensor_Model(m_i, x_t, z_t) - l_0
    else
        l_{t,i} = l_{t-1,i}
    end if
end for
return {l_{t,i}}
```

Inverse Sensor Model

Algorithm Inverse_Range_Sensor_Model(m_i, x_t, z_t)

Let (x_i, y_i) be the center-of-mass of m_i
$r = \sqrt{(x_i - x)^2 + (y_i - y)^2}$
$\phi = \text{atan2}(y_i - y, x_i - x) - \theta$
$k = \arg\min_j |\phi - \theta_{j,\text{sens}}|$
if $r > \min(z_{\max}, z_t^k + \alpha/2)$ or $|\phi - \theta_{k,\text{sens}}| > \beta/2$ **then**
    return l_0
else if $z_t^k < z_{\max}$ and $|r - z_t^k| < \alpha/2$ **then**
    return l_{occ}
else if $r \leq z_t^k$ **then**
    return l_{free}
end if

Scan Matching, Odometry and Occupancy Grids

- *Deterministic Occupancy Grids*
- *Updating Deterministic Occupancy Grids in the Ideal Case*
- *Probabilistic Occupancy Grids*
- *Updating Probabilistic Occupancy Grids with Noisy Sensors*
- **Scan Matching, Odometry and Occupancy Grids**
- *Particle Filters and FastSLAM*
- *Suboptimal Particle and Factor Graph Approaches*

Mapping vs. SLAM

- So far we've just looked at the mapping problem,

$$f\left(O \middle| \mathbf{x}_{0:k}, \mathbf{z}_{1:k}\right)$$

- This all assumes the robot pose is *known* and *we don't need knowledge* of the control inputs

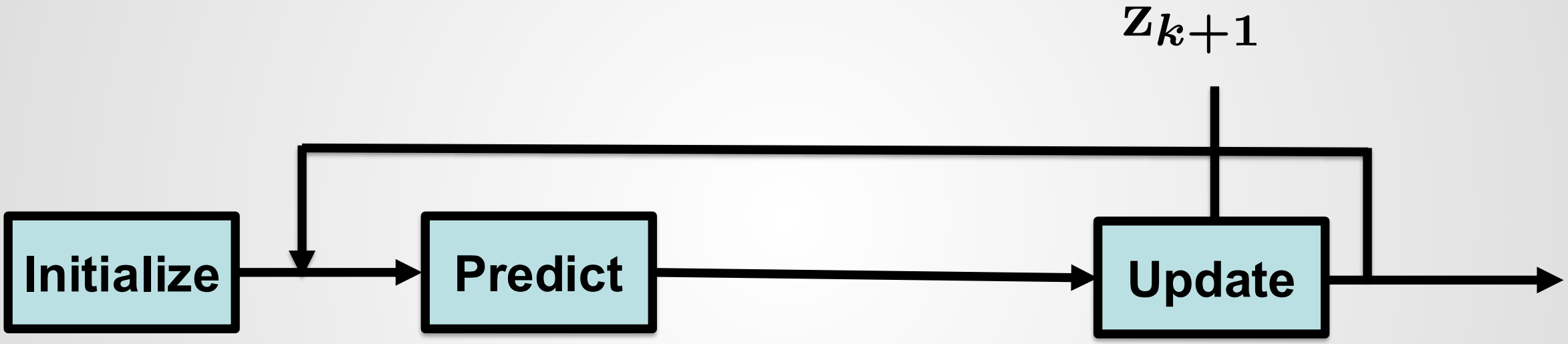
Grid SLAM

- To move to a SLAM solution, we need to estimate both the platform poses and the map *at the same time*,

$$\Pr\left(\mathbf{x}_k, O \middle| \mathbf{x}_0, \mathbf{u}_{1:k}, \mathbf{z}_{1:k}\right)$$

- To do this, we'll use our familiar recursive structure which includes the platform prediction

Probabilistic Occupancy Grid Mapping

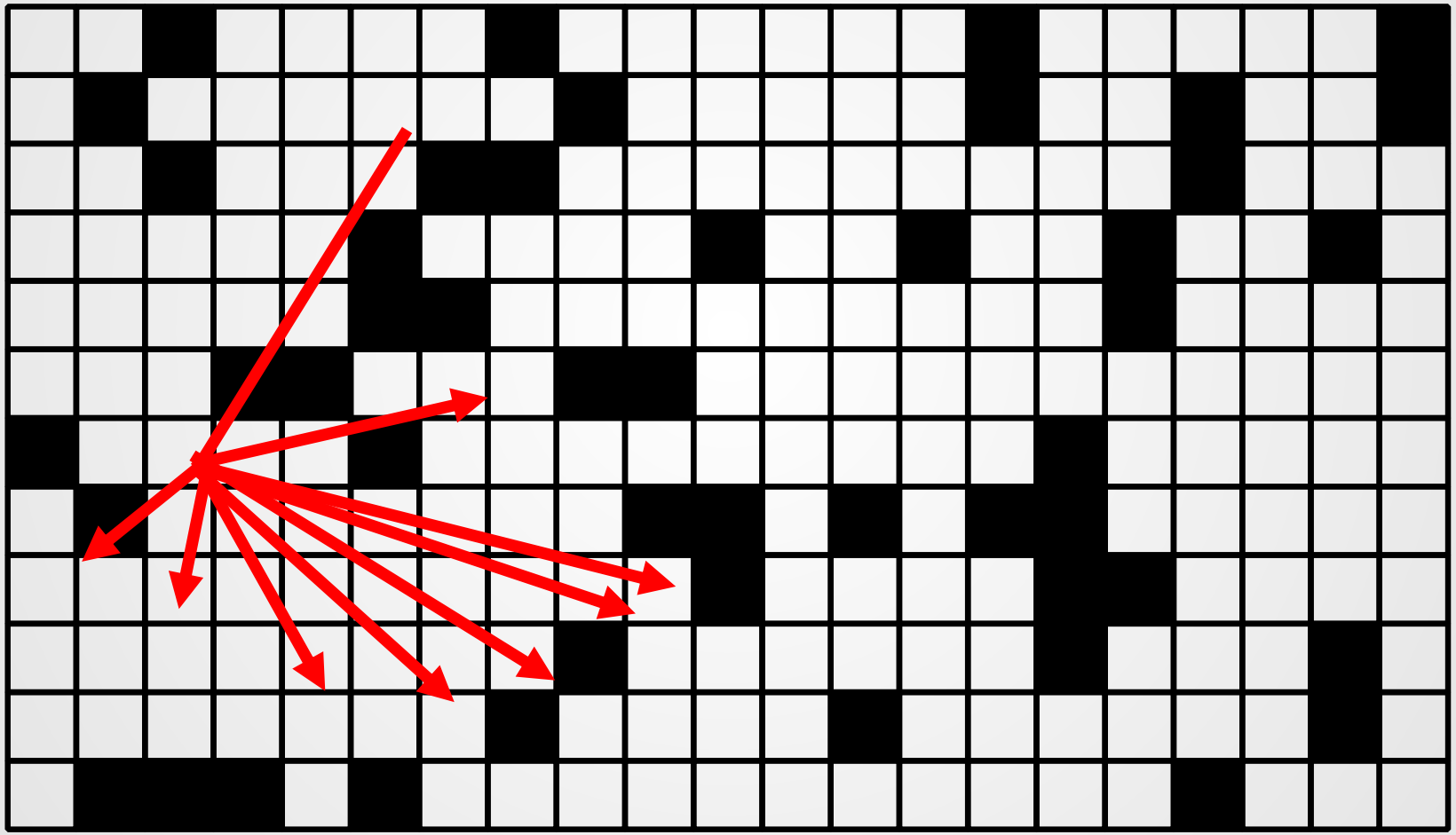


$\Pr(O|\mathbf{x}_0)$ $\Pr(\mathbf{x}_{1:k+1}, O|\mathbf{x}_0, \mathbf{u}_{1:k+1}, \mathbf{z}_{1:k})$ $\Pr(\mathbf{x}_{1:k+1}, O|\mathbf{x}_0, \mathbf{u}_{1:k+1}, \mathbf{z}_{1:k+1})$

Scan Matching

- Scan matching is used to predict / refine the estimated platform pose before the sensor data is fused
- We can use priors from vehicle kinematics / encoders to provide an initial estimate
- However, we often refine this estimate by aliging the scan with the existing map

Scan Matching



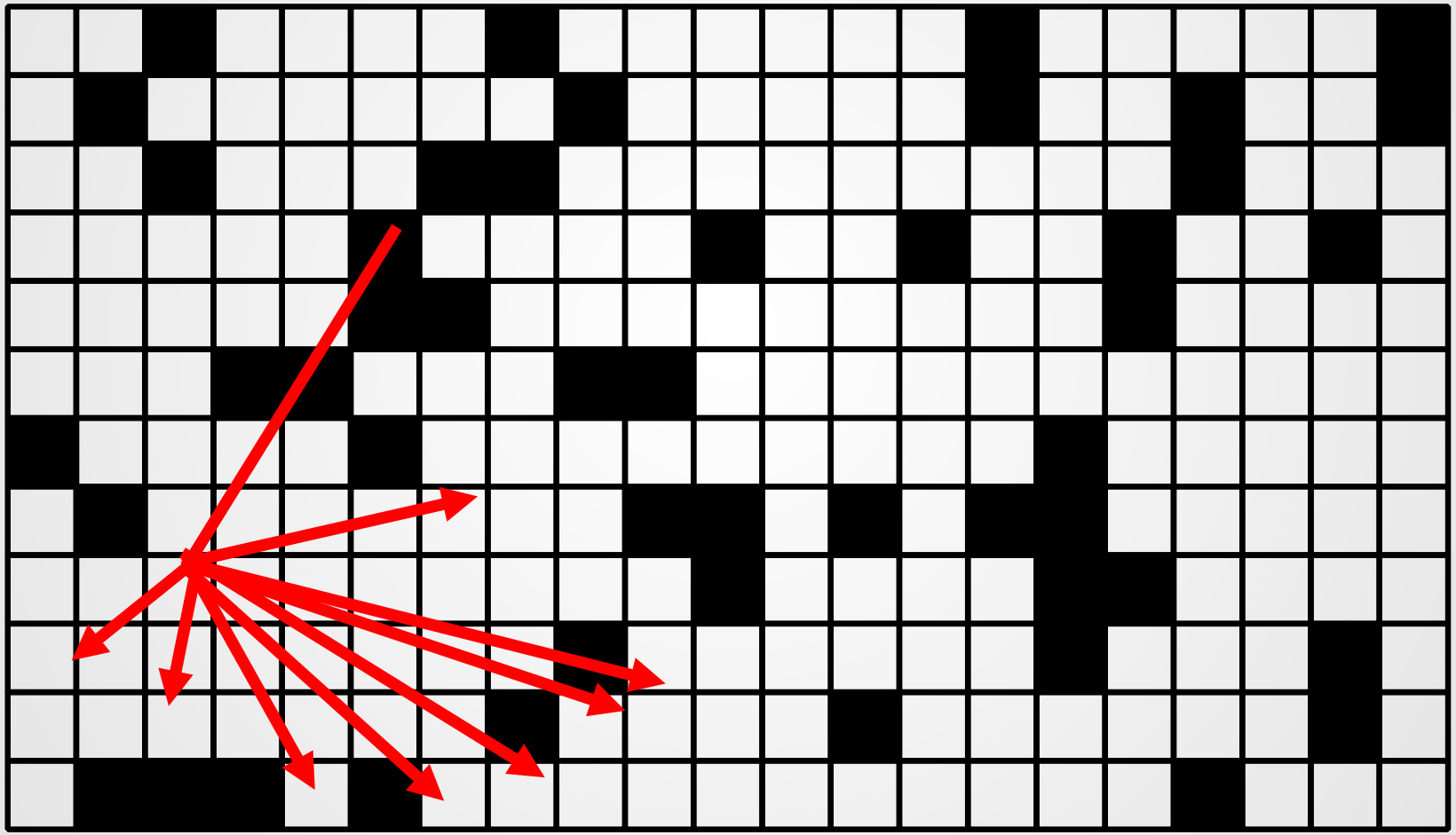
Scan Matching

- We seek the pose of the vehicle which maximizes the likelihood,

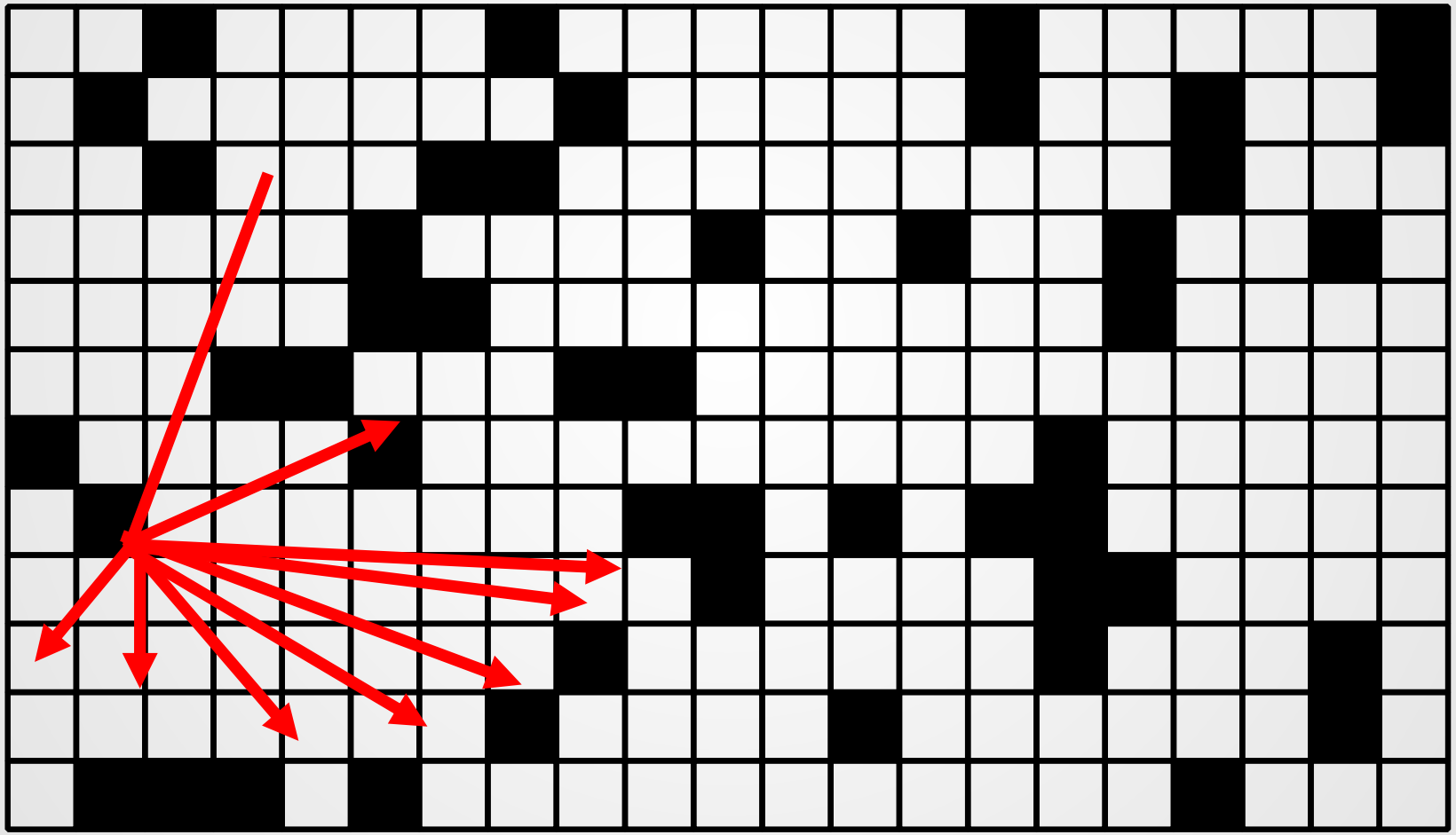
$$\mathrm{x}_k = \arg\max_{\mathrm{x}} f\left(\mathrm{z}_k | \mathrm{x}, O\right)$$

- This is an another optimization problem
- Like all the other problems we iteratively refine it

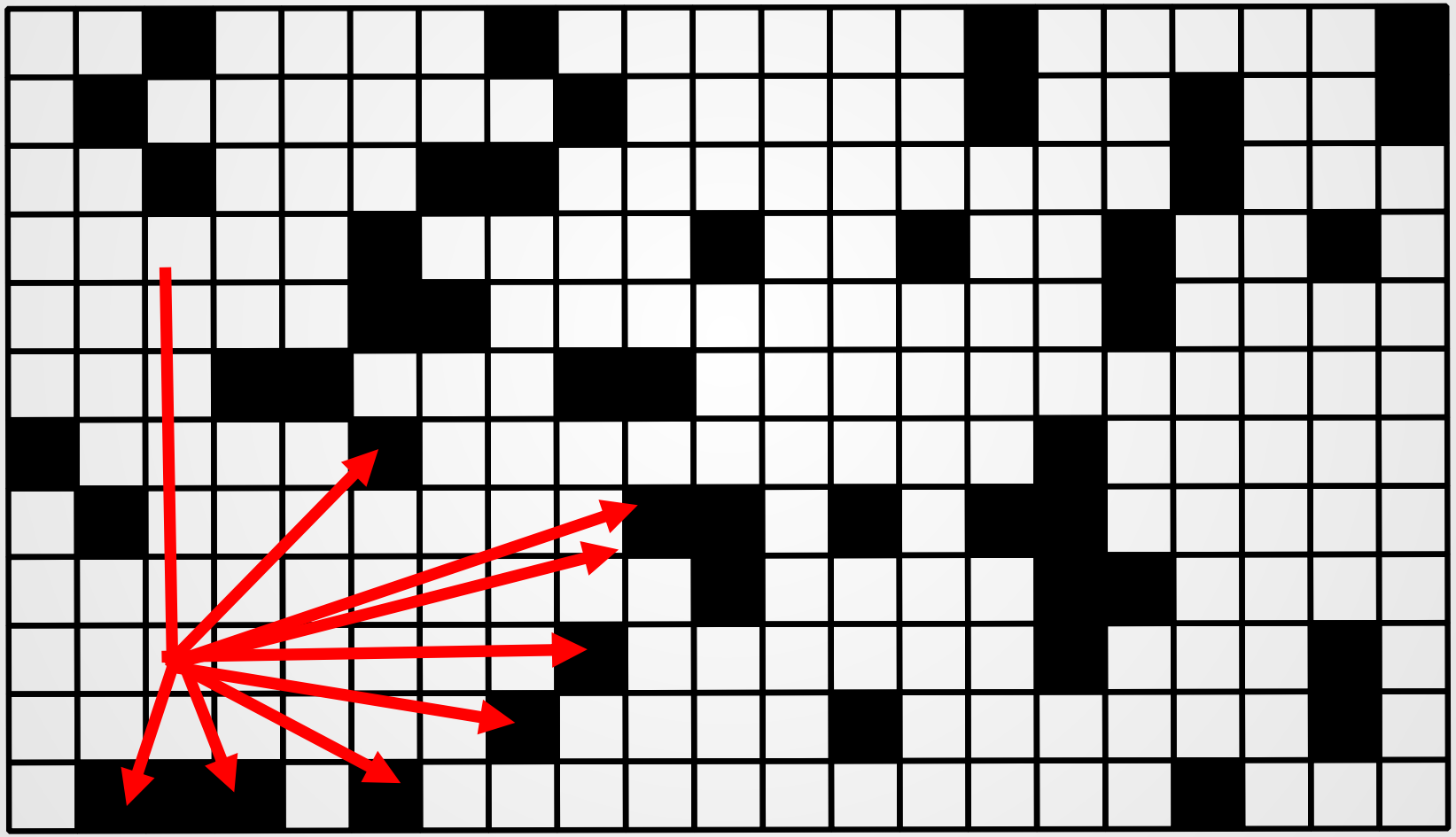
Scan Matching



Scan Matching



Scan Matching



Scan Matching with Occupancy Grids

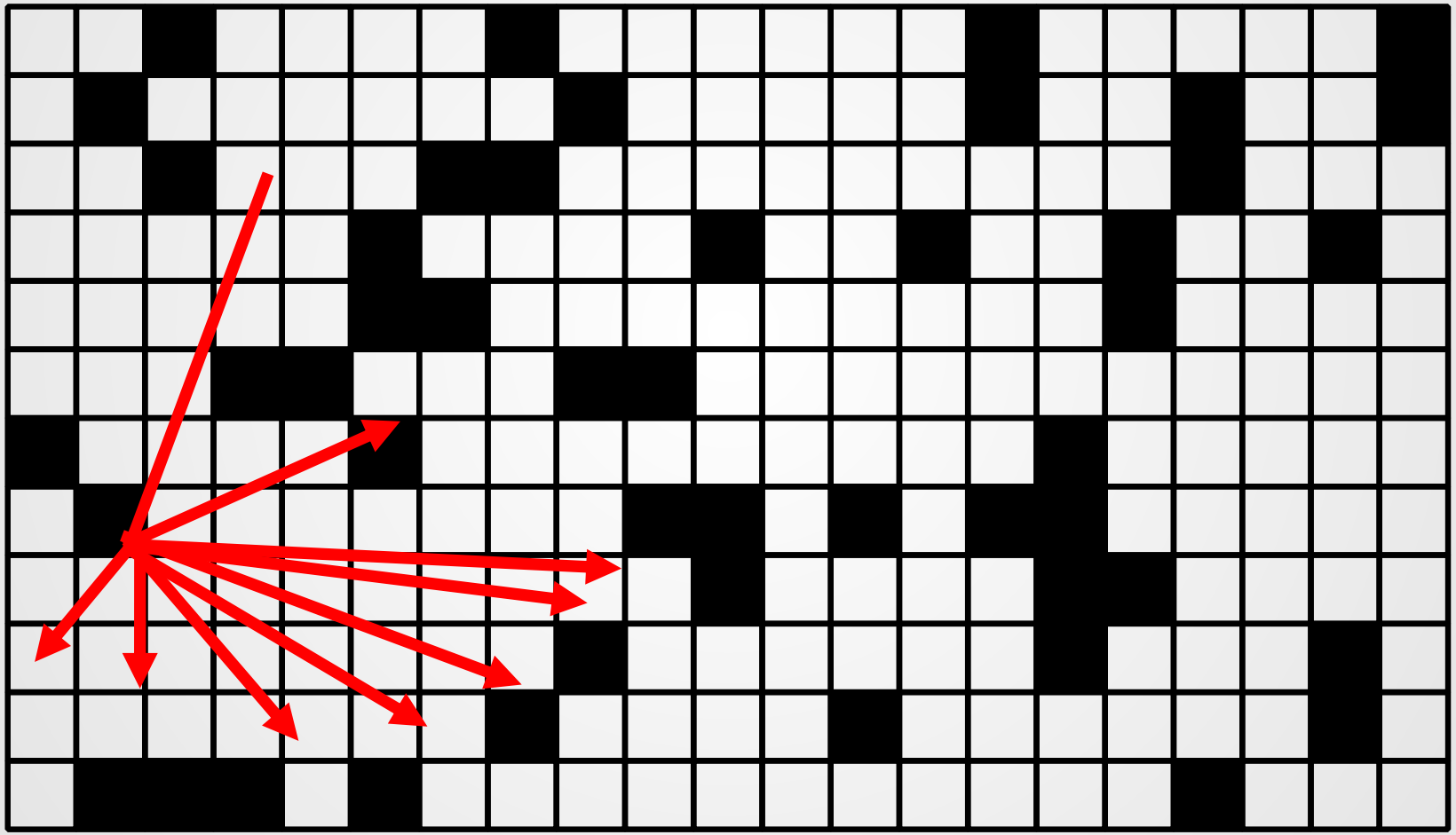
- To carry out scan matching, some kind of cost or score of the quality of fit has to be made
- A very common approach is to use a *likelihood field*
- Likelihood fields match estimated obstacles in the scan and the map with one another

Likelihood Field Preprocessing

- **Extract occupied cells from the map:**
  - **These have occupancy probability greater than, say, 0.5**

- **Identify laser scans which potentially detected a surface:**
  - **Remove all the scans whose detected range is the maximum**

Scan Matching



Likelihood Field

- Given a candidate pose:
  - Project the end points of all the laser scan rays into world coordinates using the current estimated pose
  - For each point, work out the closest occupied cell in the map
  - The cost of a configuration is the sum of the cost terms

Likelihood Field Approach

Occupancy grid map

Likelihood field

Particle Filters and FastSLAM

- *Deterministic Occupancy Grids*
- *Updating Deterministic Occupancy Grids in the Ideal Case*
- *Probabilistic Occupancy Grids*
- *Updating Probabilistic Occupancy Grids with Noisy Sensors*
- *Scan Matching, Odometry and Occupancy Grids*
- **Particle Filters and FastSLAM**
- *Suboptimal Particle and Factor Graph Approaches*

Handling Uncertain Platform Positions

- Although scan matching can reduce odometry errors, errors still arise
- Therefore, we need to model the uncertainty in the platform pose
- Although this is easy to do with sparse features, with occupancy grids it's tough to do, even with the independence assumptions
- Rather, we use refactorization with particle filters

Refactoring GridSLAM

- **We can refactor the joint probability into two terms**

$$f\left(\mathbf{x}_{1:k}, O \middle| \mathbf{x}_0, \mathbf{u}_{1:k}, \mathbf{z}_{1:k}\right) = f\left(O \middle| \mathbf{x}_{0:k}, \mathbf{z}_{1:k}\right) \times f\left(\mathbf{x}_{1:k} \middle| \mathbf{x}_0, \mathbf{u}_{1:k}, \mathbf{z}_{1:k}\right)$$

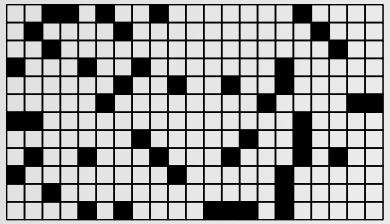
First Term

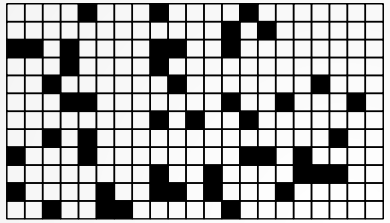
- The first term is

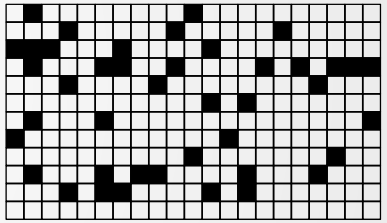
$$f\left(O \middle| \mathbf{x}_{0:k}, \mathbf{z}_{1:k}\right)$$

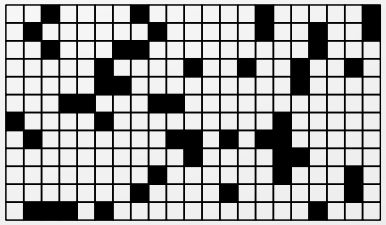
- This is the term for building a probabilistic occupancy grid, *given that the trajectory of the platform is known*
- Therefore, we can use the earlier results to update it

Visualizing the Factorization









Second Term

- The second term is

$$f\left(\mathbf{x}_{1:k} \middle| \mathbf{x}_0, \mathbf{u}_{1:k}, \mathbf{z}_{1:k}\right)$$

- This is the probability distribution over the pose of the platform
- This is where the platform uncertainty is accounted for

Implementing the Vehicle Pose Factorization

- The factorization is normally implemented using *a particle filter*
- This approximates the probability distribution by a set of particles

$$\mathbf{x}_k^{[i]} \sim f\left(\mathbf{x}_k\right)$$

- Each particle is a sample in the state space
- You can think of it as a hypothesis that “this is correct pose”

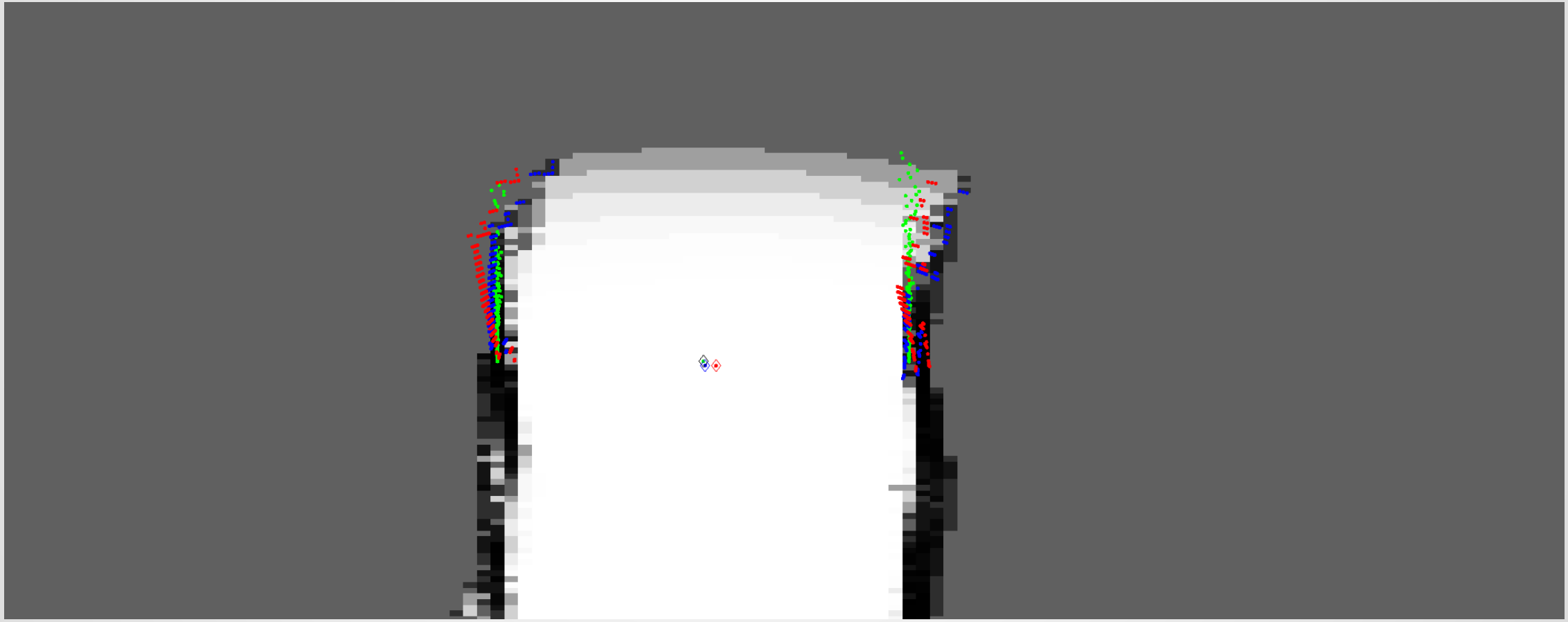
Implementing the Vehicle Pose Factorization

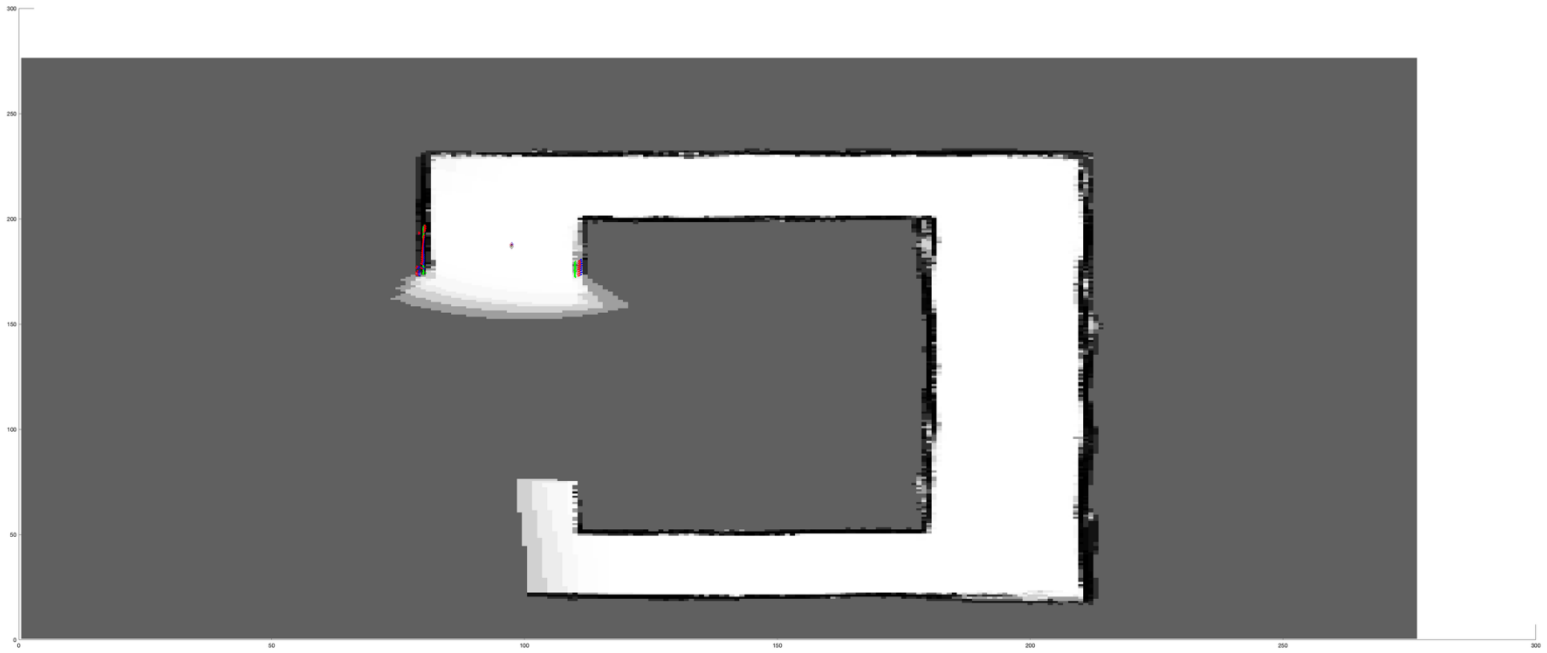
- For each particle we have to specify the transition kernel and the likelihood
- The transition kernel can come from the platform process model or can exploit information about subsequent measurements
- The likelihood model measures the compatibility between the sensor measurement and the map built for this particle

300
250
200
150
100
50
0
0
50
100
150
200
250

Partially Built Map

Partially Built Map (Zoomed In)




300
250
200
150
100
50
0
0
50
100
150
200
250
300

Limitations

- It feels like this should give good results at the cost of scalability
- However, the maps can vary a lot for each particle
- This leads to the ESS becoming very small
- This leads to a lot of duplication of the map
- Since the map isn’t randomized by process noise you end up with all the particles using the same map

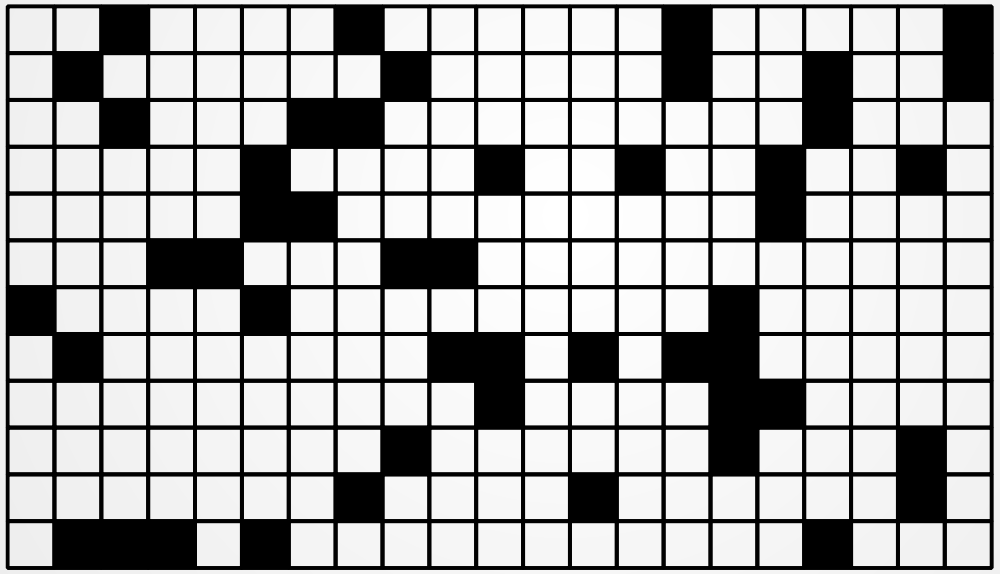
Suboptimal Particle and Factor Graph Approaches

- *Deterministic Occupancy Grids*
- *Updating Deterministic Occupancy Grids in the Ideal Case*
- *Probabilistic Occupancy Grids*
- *Updating Probabilistic Occupancy Grids with Noisy Sensors*
- *Scan Matching, Odometry and Occupancy Grids*
- *Particle Filters and FastSLAM*
- **Suboptimal Particle and Factor Graph Approaches**

Shared Maps

- The standard way to address the issue with particle filters is to use a single shared map
- In this system:
  - There is a single occupancy grid map
  - The platform poses are represented using a particle filte

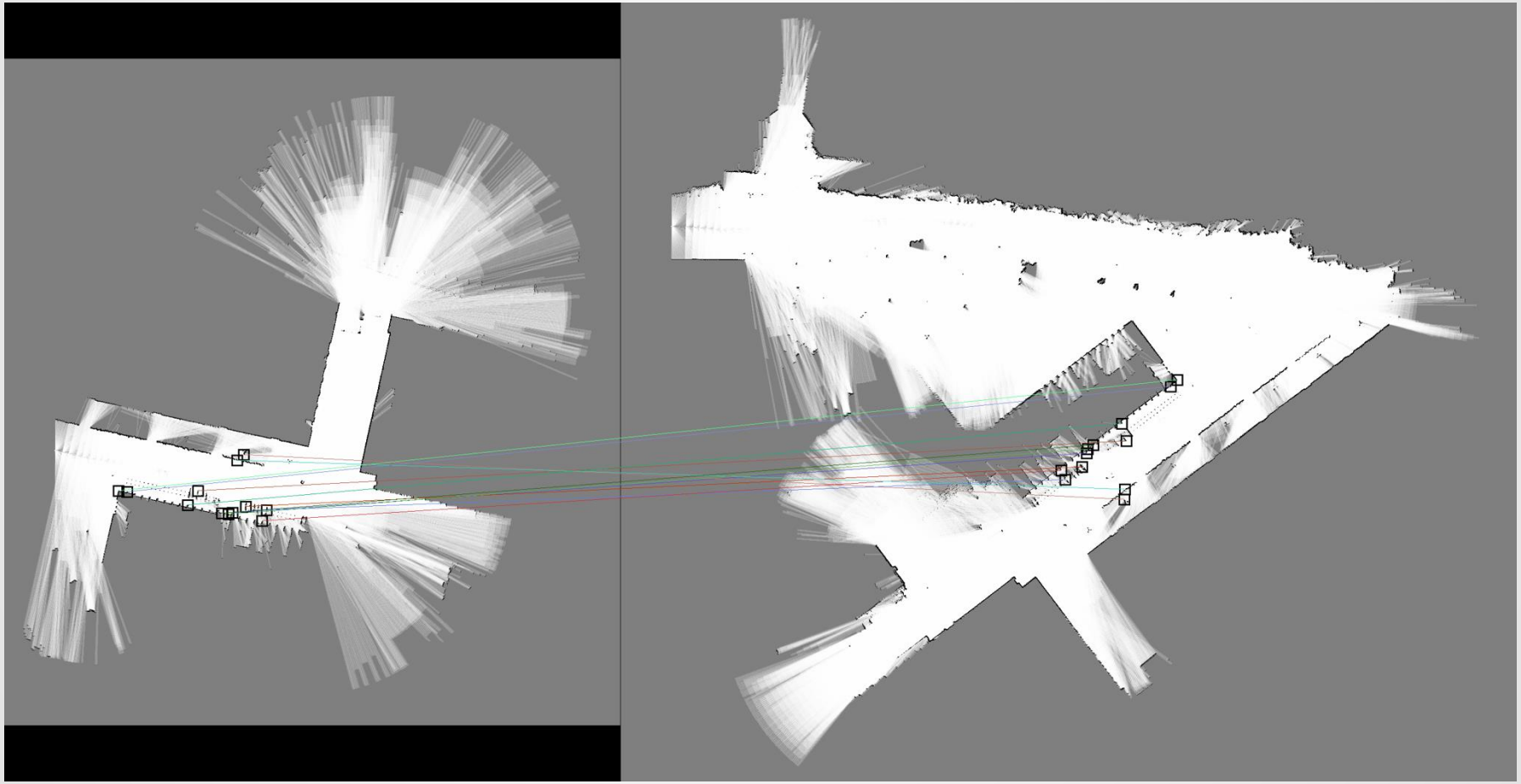
Shared Map



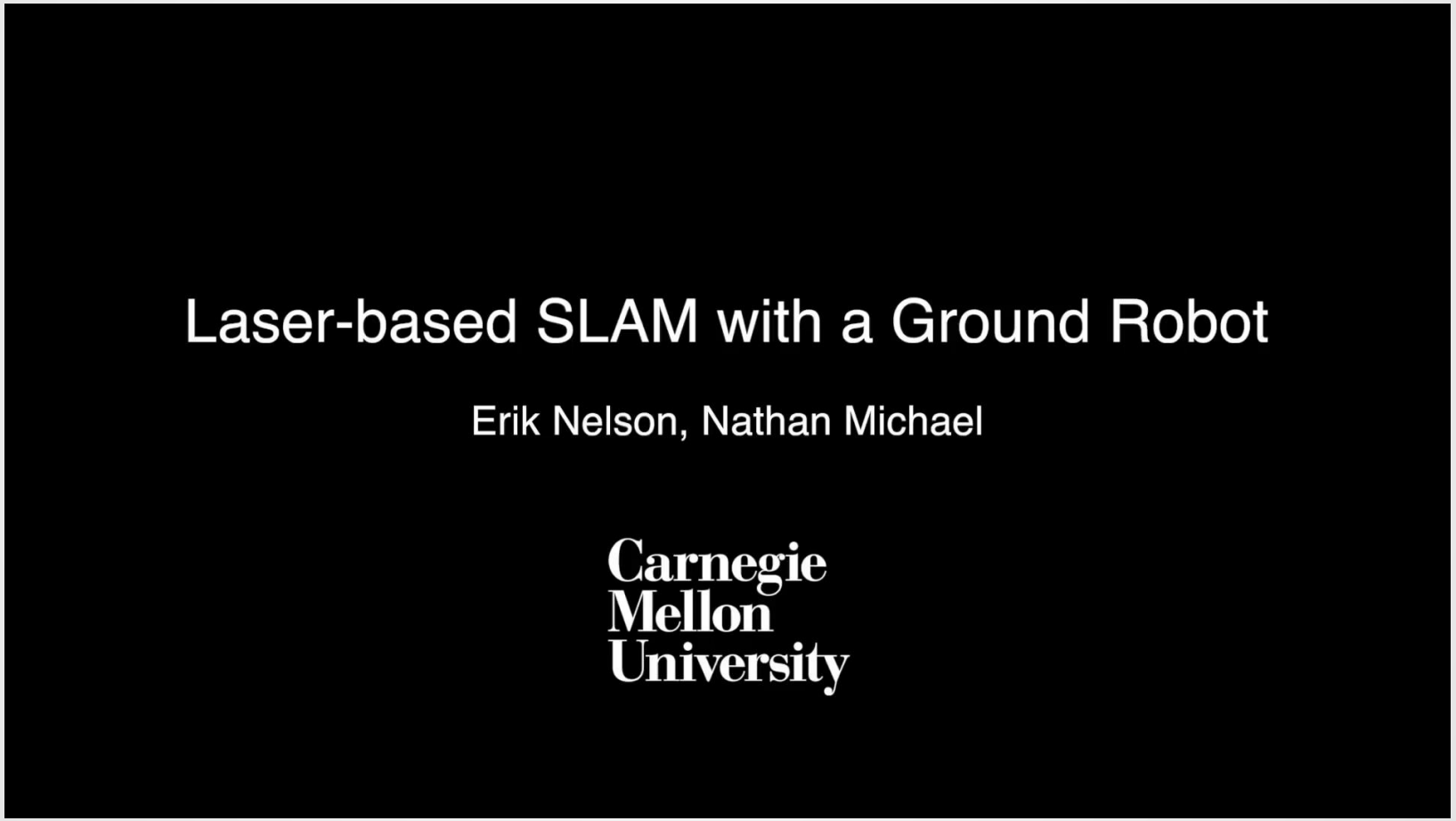
Shared Maps

- Before resampling, we figure out the particle with the highest weight
- This is assumed to be the “most accurate” estimate
- We then update the shared map assuming the pose of the “best” estimate is perfect

Probabilistic Occupancy Grids



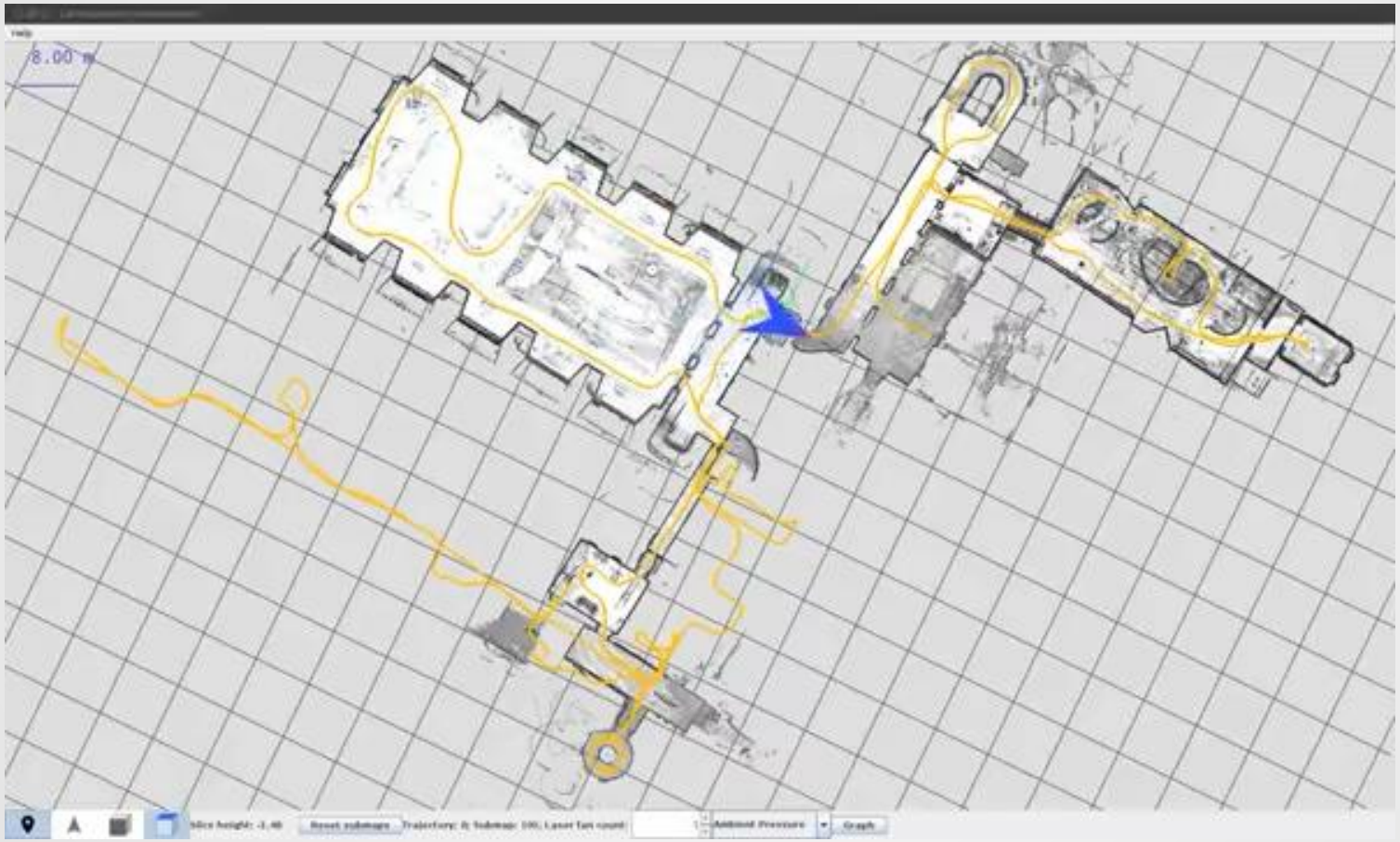
SLAM and Indoor Spaces



Factor Graph Based Approaches

- It is also possible to use a factor graph to carry out occupancy grid matching
- The approach is very similar to the point cloud:
  - A submap is locally built using occupancy grids assuming pose is perfect
  - Each submap is then frozen
  - The submaps are aligned with one another similar to ICP in an occupancy grid

Cartographer



Summary

- Occupancy grids are much more difficult to work with than sparse features
- With sparse features, the optimal solution can be implemented
- With occupancy grids, the dependency between the cells means that the approach is impractical, even with highly simplified systems
- Therefore, various approaches with differing degrees of being principled are used